

Designing a Diagnostic-Reasoning Serious Game for Point-of-Sale Technical Support: A Data-Driven Simulation Architecture with a Contained Language-Model Customer

Final Year Project — Academic Report

Student	<i>[Student Name]</i>
Student ID	<i>[Student ID]</i>
Programme	<i>[Programme / Award]</i>
Supervisor	<i>[Supervisor Name]</i>
Submission date	August 2026
Artefact	<i>POS Tech Support — Unity 6 (6000.5.4f1) simulation game</i>
Word count	≈ 16,200 (Chapters 1–8, including tables; references and appendices excluded)
Referencing style	Harvard

Abstract

Technical-support work is a troubleshooting profession, yet the skill at its centre — reasoning backwards from an unreliable verbal report to a root cause inside a dependent system — is rarely trainable outside live service desks, where mistakes are expensive and reproducible practice is impossible. This project designs and implements *POS Tech Support*, a single-player simulation game in which the player works night shifts on a Point-of-Sale (POS) support line: answering calls from non-technical shopkeepers, verifying the caller's identity against a customer-relationship-management (CRM) record, connecting to a simulated remote desktop, tracing a fault through a dependency graph of coupled components, and closing the ticket.

The report makes three contributions. First, it presents a fault-modelling design in which failures are *invalid states inside simulated modules* rather than scripted boolean flags, and in which faults propagate downstream through an explicit dependency graph while repairs must proceed upstream — a game-mechanical realisation of the device-model component of Jonassen and Hung's (2006) troubleshooting architecture. A taxonomy of forty authored faults is built on an explicit discriminability rule: every fault must be separable from a fault the player already knows, and the report documents for each one which prior fault it is confusable with. The same rule is applied to identity rather than only to faults: the CRM directory the player verifies against is generated in families of mistakeable shop names, and each site's network and record-store identity is derived from its own name, so authored fault content is store-agnostic by construction. Second, it presents a four-layer software architecture (Data → Simulation → Logic → AI) in which all authored content lives in Unity `ScriptableObject` assets and all mutable state lives in ordinary C# classes, so that verdict logic is a pure function of current state and can never drift out of date. Third, it presents a containment architecture for a language-model non-player character: the simulated customer is handed a deliberately impoverished `GroundTruth` view containing only lay symptom descriptions and identity claims, a rule-based `DialoguePolicy` decides *what may be said*, an optional locally hosted small model may only *reword* that decision, and a `GroundingGuard` inspects the result before display. The model is therefore structurally incapable of leaking the answer, because it was never given it — a stronger guarantee than output filtering alone (Rebedea *et al.*, 2023).

The artefact comprises approximately 7,600 lines of C# across fifty files, a 1,465-line design specification, and a validated 2,700-line web prototype used as an executable reference. Verification was performed by a cascade smoke test that injects all forty authored faults and asserts the resulting Blocked/Error pattern, together with structured playtesting in the editor. Limitations are reported honestly: no automated unit-test suite exists, the cross-night recurrence mechanism is wired but dormant, voice interaction was descope, and no summative study with human participants was conducted, so all learning claims in this report are design claims argued from literature rather than measured effects.

Keywords: serious games; troubleshooting; diagnostic reasoning; model-based diagnosis; game architecture; `ScriptableObject`; large language models; NPC dialogue; guardrails; Unity.

Contents

1. Introduction	1
1.1 Background and context	1
1.2 Problem statement	1
1.3 Aim and objectives	2
1.4 Scope and delimitations	2
1.5 Report structure	3
2. Literature Review	4
2.1 Serious games and simulation-based training	4
2.2 Learning to troubleshoot	4
2.3 Fault modelling and model-based diagnosis	5
2.4 Architecture for data-driven game systems	6
2.5 Language-model non-player characters and the containment problem	6
2.6 Comparable games	7
2.7 Research gap and positioning	7
3. Methodology	8
3.1 Research paradigm: Design Science	8
3.2 Specification-first development	8
3.3 Prototype-then-port	9
3.4 Milestone-driven build order	9
3.5 Tools and environment	10
3.6 Evaluation approach	10
4. Design	12
4.1 The core loop	12
4.2 Seven invariant design principles	12
4.3 Four-layer architecture	13
4.4 The POS domain model	13
4.5 Fault taxonomy: forty faults and the discriminability rule	14
4.6 The dependency cascade: Blocked versus Error	16
4.7 Resolution semantics	16
4.8 Verification and the noisy human sensor	17
4.9 Difficulty progression and fading guidance	18
4.10 Campaign, consequence and economy	19
5. Implementation	20
5.1 Structure and scale	20
5.2 Data layer, and a deliberately uncomfortable trade-off	20
5.3 Simulation layer	21
5.4 Logic layer	22
5.5 Services and the composition root	23
5.6 The AI layer: containment in code	23

5.7 User interface	25
5.8 Content authoring pipeline	26
6. Testing and Evaluation	27
6.1 Verification strategy and its honest limits	27
6.2 Cascade verification over all forty faults	27
6.3 Specification coverage audit	28
6.4 Discriminability audit	29
6.5 Containment evaluation of the dialogue agent	30
6.6 Known limitations and dormant mechanics	30
6.7 Threats to validity	31
7. Discussion	32
7.1 The artefact against the troubleshooting-instruction literature	32
7.2 Architecture as pedagogy	33
7.3 Policy-as-brain: a pattern with reach beyond games	33
7.4 Reflection on the specification-first method	34
7.5 Practical and ethical considerations	34
8. Conclusion and Future Work	36
8.1 Conclusion	36
8.2 Future work	37
9. References	39
10. Appendices	42
Appendix A — The forty-fault corpus	42
Appendix B — Modules and application mapping	43
Appendix C — Services and single responsibilities	44
Appendix D — Artefact metrics	44
Appendix E — Glossary	45

1. Introduction

1.1 Background and context

Point-of-Sale systems are the operational nervous system of small retail and hospitality businesses. A single till lane couples an operating system, POS application software, a card-payment terminal, a receipt printer, a cash drawer, a local network and a transaction database — components supplied by different vendors, configured by different people, and understood in full by almost nobody on site. When a lane stops working, the person who notices is a shop owner or a counter staff member with no technical vocabulary, and the person who must fix it is a remote support agent who cannot see the room.

That agent's work is *troubleshooting* in the precise sense used by the instructional-design literature: a constrained diagnostic problem in which a system's observed behaviour deviates from its expected behaviour, the cause is hidden, and the solver must generate and test hypotheses against a mental model of how the system's parts depend on one another (Jonassen and Hung, 2006). It is also, in practice, a communication problem: the primary sensor available to the agent is a stressed, non-technical human who describes symptoms in the wrong words, attributes causes incorrectly, and misremembers identifying details.

Training for this work is structurally difficult. Live service desks cannot manufacture reproducible faults on demand; a trainee cannot be handed a printer whose driver is corrupted *and* whose cash drawer occupies the same serial port, twice, to compare. Documentation-based training conveys device knowledge but not the experiential and strategic knowledge that distinguishes competent troubleshooters, which is exactly the gap Jonassen and Hung (2006) identify in conventional instruction. Simulation is the standard response to this class of problem, and the meta-analytic evidence for it is reasonably strong: Sitzmann (2011), pooling sixty-five studies and 6,476 participants, found that trainees taught with computer-based simulation games showed declarative knowledge 11% higher, procedural knowledge 14% higher, retention 9% higher and post-training self-efficacy 20% higher than comparison groups.

1.2 Problem statement

Existing simulation games rarely model the *dependency structure* that makes technical diagnosis hard. Commercial "job simulator" titles typically implement faults as scripted flags with scripted fixes, which trains recall of a lookup table rather than inference. Conversely, the growing body of work on language-model-driven non-player characters (Peng *et al.*, 2024) offers convincing conversational partners but introduces a specific hazard for a diagnostic game: a model that knows the answer will eventually give it away, whether through hallucination, sycophancy, or a player who simply asks the right question. Both failure modes destroy the learning object — the first by removing the reasoning, the second by removing the puzzle.

The problem this project addresses is therefore: **how can a simulation game represent faults in a coupled technical system such that diagnosis requires genuine inference, and simultaneously use a language model to make the reporting human convincingly unhelpful without allowing that model to leak the solution?**

1.3 Aim and objectives

Aim. To design, implement and critically evaluate a playable simulation game that develops diagnostic reasoning for POS technical support, using a state-based fault model with explicit dependency propagation and a structurally contained language-model customer.

Objectives.

1. **O1 — Domain model.** Model the POS ecosystem as coupled modules with an explicit dependency graph, distinguishing failures *caused by* an upstream component from failures *local to* a component, and justify why that distinction is pedagogically load-bearing rather than cosmetic.
2. **O2 — Fault taxonomy.** Author a corpus of faults large enough to sustain extended play, subject to a discriminability constraint: each fault must be separable, by observable evidence, from at least one fault the player has already met.
3. **O3 — Software architecture.** Specify and implement an architecture that separates authored content from runtime state, keeps verdict evaluation a pure function of current state, and permits content to be added without code changes.
4. **O4 — Contained conversational agent.** Design a dialogue pipeline in which a rule-based policy decides content and an optional local language model only phrases it, and in which the model is never given the information it could leak.
5. **O5 — Verification.** Establish a verification strategy adequate to the artefact's scale, and report the resulting evidence — including where it is thin — without overclaiming.
6. **O6 — Critical evaluation.** Evaluate the design against the troubleshooting-instruction literature and against the constraints of the chosen platform, and identify what would be required to make an empirical learning claim.

1.4 Scope and delimitations

The artefact is a single-player desktop game built in Unity 6 (6000.5.4f1). Its content is entirely in English, including all customer dialogue, on the grounds that a mixed-language corpus would compromise both the misnaming mechanic and the behaviour of small English-first language models.

Four delimitations are stated explicitly and are revisited in §6.6:

- **No human-subjects study.** No summative experiment measuring learning transfer was conducted. Every claim about learning in this report is a *design* claim, argued from the literature, and is labelled as such.
- **Voice interaction descoped.** Speech-to-text and text-to-speech (milestone M7 in the design specification) were designed for but not implemented; the dialogue layer is input-agnostic so that they can be added without redesign.
- **Cross-night consequence is dormant.** The mechanism by which a symptomatic-but-incomplete repair recurs on a later night is fully implemented but never fires with the current content corpus, for a reason documented in §6.6.
- **Content is authored, not procedurally generated.** Variety comes from combining authored faults with randomised store, persona and caller-authorisation state, not from generating novel faults.

1.5 Report structure

Chapter 2 reviews the relevant literature across four bodies of work: serious games and simulation-based training; troubleshooting and diagnostic reasoning; model-based diagnosis in artificial intelligence; and language-model agents with their containment problem. Chapter 3 sets out the Design Science Research methodology, the specification-first process, and the milestone-driven build order. Chapter 4 presents the design of the artefact, organised around seven invariant principles. Chapter 5 documents the implementation, including the points at which it deliberately departs from the specification. Chapter 6 reports verification and evaluation, including negative results. Chapter 7 discusses the artefact against the literature and reflects on the method. Chapter 8 concludes and sets out future work.

2. Literature Review

2.1 Serious games and simulation-based training

The term *serious game* predates digital media: Abt (1970) applied it to games with "an explicit and carefully thought-out educational purpose" that are "not intended to be played primarily for amusement". Its modern currency derives from Prensky (2001), Gee (2003) and Michael and Chen (2006), whose common argument is that well-designed games instantiate learning principles — clear goals, graduated challenge, immediate feedback, safe failure — that formal instruction frequently fails to deliver.

The empirical picture is positive but modest, and it matters for this project that the modesty be stated. Wouters *et al.* (2013), meta-analysing seventy-seven learning studies ($N = 5,547$) and thirty-one motivation studies ($N = 2,216$), found serious games more effective than conventional instruction for learning ($d = 0.29$, $p < 0.01$) and retention ($d = 0.36$, $p < 0.01$), but *not* significantly more motivating ($d = 0.26$, $p > 0.05$). This is a useful corrective to the common assumption that a game is pedagogically justified by engagement alone. Sitzmann's (2011) results, cited in §1.1, are stronger, and the difference is instructive: Sitzmann's corpus concerns *simulation* games used in workplace training, where the game's model of the task closely mirrors the task itself. Clark, Tanner-Smith and Killingsworth (2016) reach a compatible conclusion from a systematic review, finding that outcomes depend far more on specific design decisions than on the medium.

The mechanism by which games teach is generally theorised in terms of an iterative judgement–feedback cycle. Garris, Ahlers and Driskell (2002) model game-based learning as a loop of user judgements, behaviour and system feedback that must be *debriefed* to yield learning outcomes; Kolb (1984) frames the same cycle as concrete experience, reflective observation, abstract conceptualisation and active experimentation. Both imply a design requirement that this project takes seriously: the game must make the *consequences* of a diagnostic decision legible, not merely register success or failure. Malone (1981) and later Ryan, Rigby and Przybylski (2006) add the motivational account — challenge calibrated to competence, curiosity, and player autonomy — while Csikszentmihalyi (1990) supplies the flow argument for progressive difficulty. Sweller's (1988) cognitive-load theory supplies the corresponding warning: a simulation that presents its full complexity immediately spends the learner's working memory on interface comprehension rather than on diagnosis, which is the direct justification for the staged content unlock and fading guidance described in §4.9.

2.2 Learning to troubleshoot

The single most directly relevant work is Jonassen and Hung's (2006) design architecture for troubleshooting instruction. They characterise troubleshooting as the most common form of everyday professional problem solving, and argue that conventional instruction fails because it teaches either system theory *or* diagnostic procedure, neither of which transfers. Competent troubleshooting, they contend, requires the integration of three knowledge types:

- **Domain/conceptual knowledge** — how the class of system works in principle;
- **Device knowledge** — how *this* system's components are connected and what each does, i.e. a runnable mental model;

- **Experiential/strategic knowledge** — accumulated case memory and search strategy: what usually goes wrong, what to test first, how to bisect a fault space.

Their prescription is a learning environment in which the learner must generate and test a hypothesis for every action taken, relate every action to a conceptual model of the system, and consult experienced troubleshooters. Mapping these three requirements onto game mechanics is the pedagogical spine of Chapter 4: the dependency graph supplies device knowledge, the diagnostic action/clue structure forces hypothesis-before-action, the authored knowledge base substitutes for the experienced colleague, and the fault corpus builds case memory.

Adjacent cognitive-science work explains *why* device knowledge is the hard part. Chi, Feltovich and Glaser (1981) showed that experts and novices categorise problems differently — experts by underlying principle, novices by surface feature — which predicts precisely the error a POS support trainee makes: classifying "the receipt is wrong" as a printer problem because printers produce receipts. Gentner and Stevens (1983) and Norman (1983) establish that people reason about devices through mental models that are typically incomplete and unstable. Rasmussen's (1983) skills-rules-knowledge framework locates novice diagnosis at the effortful knowledge-based level and expert diagnosis at the rule-based level, and Klein's (1998) recognition-primed decision model describes the resulting expert behaviour: pattern recognition first, deliberation only when the pattern fails. Ericsson, Krampe and Tesch-Römer (1993) supply the practice condition — repeated, feedback-rich, difficulty-calibrated repetition of the discriminating cases — which a game can provide and a live service desk cannot. Reason's (1990) taxonomy of human error, finally, motivates a mechanic that most job simulators omit: the possibility of making things *worse*, and the distinction between a repair that removes a symptom and a repair that removes a cause.

2.3 Fault modelling and model-based diagnosis

Artificial-intelligence research on diagnosis provides the formal vocabulary for the fault model in §4.6. Reiter's (1987) theory of diagnosis from first principles defines a diagnosis as a minimal set of components whose assumed abnormality reconciles a system description with observed behaviour — reasoning from a *model of correct structure and behaviour* rather than from a table of symptom-cause pairs. de Kleer and Williams (1987) extend this to multiple simultaneous faults, and in doing so identify the phenomenon this game exploits most heavily: when several components are faulty at once, the observable symptoms of one can mask another entirely.

Two consequences shape the design. First, if faults are represented as invalid *states* of modelled components rather than as scripted outcomes, then symptom propagation is derivable rather than authored, and novel fault combinations produce coherent behaviour without additional authoring. Second, masking is a first-class phenomenon rather than a bug: a network outage genuinely hides a printer driver fault downstream of it, and a diagnostic environment that models this teaches something a symptom-table environment cannot — that clearing a blocker is the beginning of a diagnosis rather than the end. The game's `Latent → Active` fault-state transition (§4.6) is a direct implementation of this insight.

The industrial framing comes from IT service management. ITIL 4 (AXELOS, 2019) separates *incident* management, whose objective is restoring service quickly, from *problem* management, whose objective is eliminating underlying causes. This distinction is not merely administrative: it is the source of the game's most interesting scoring decision, the separation of `symptomCleared` from `rootCauseFixed` (§4.7), which encodes the fact that a support agent can satisfy a customer and still have failed.

2.4 Architecture for data-driven game systems

Because a core objective is that content be extensible without code changes, the project draws on the software-architecture literature. Gamma *et al.* (1994) supply Factory Method, applied in §5.4 to separate the several sources from which a ticket's fault combination may be chosen from the shared assembly of the ticket itself. Fowler (2002) and Evans (2003) supply the separation of a domain model from the services that manipulate it, which is realised here as an explicit rule that authored data assets are read-only at run-time while all mutable state lives in plain classes cloned from them. Martin's (2017) dependency rule — that inner layers must not depend on outer ones — is realised as a strict layering in which the Simulation layer knows nothing of Managers and the AI layer knows nothing of the simulated desktop.

Within Unity specifically, the relevant practice literature concerns `ScriptableObject`, a serialisable data container that lives as a project asset independent of any scene (Unity Technologies, 2025). Hipple's (2017) widely adopted treatment argues for using such assets as the primary unit of game configuration, yielding designer-editable content, reduced prefab coupling, and testability. Nystrom (2014) provides the complementary caution — that patterns imported from enterprise software can impose indirection costs disproportionate to the problem — which is why §5.2 documents a deliberate and initially uncomfortable trade-off: module state is stored as string-keyed dictionaries rather than typed fields, sacrificing compile-time safety to make faults and repairs fully data-authorable.

2.5 Language-model non-player characters and the containment problem

Language models have made conversational NPCs practical, and the research literature has moved quickly from feasibility to control. Park *et al.* (2023) demonstrated generative agents with memory, reflection and planning that produce believable emergent social behaviour. Peng *et al.* (2024), working with a games-industry team, examined player-driven emergence in LLM-driven game narrative and confronted the central design tension directly: the more freedom the model has, the less the designer can guarantee about what the player will be told. Recent work on generative NPCs has converged on prompt scaffolding and retrieval grounding as mitigations (Lewis *et al.*, 2020), while surveys of hallucination in natural language generation (Ji *et al.*, 2023) establish that fabricated-but-fluent output is an intrinsic property of the technology rather than a defect to be patched.

For a diagnostic game the risk is unusually sharp, because the failure is not merely immersion-breaking — it is *game-breaking*. If the simulated shopkeeper can be induced to say "the print spooler service has stopped", the puzzle is over. Three distinct mechanisms could produce that sentence: hallucination (Ji *et al.*, 2023), where the model invents a plausible technical detail; sycophancy, where the model agrees with a leading question from the player; and prompt injection (Greshake *et al.*, 2023), where the player deliberately manipulates the model out of character. Weidinger *et al.* (2021) situate all three within a broader risk taxonomy for language models.

The dominant engineering response is programmable guardrails: Rebedea *et al.* (2023) describe a toolkit for runtime controls that are user-defined, independent of the underlying model, and interpretable, constraining topics, dialogue flow and style. Guardrails of this kind are necessary but, for this problem, insufficient. Filtering assumes the model *possesses* the sensitive information and must be prevented from emitting it — an adversarial containment problem with no completeness guarantee. The architecture in §4.3 and §5.6 takes the stronger route: withhold the information entirely. The model receives only lay symptom descriptions and identity claims; the root cause, the technical symptom text, the diagnostic clues and the reso-

lution conditions are never placed in its context. Output filtering is retained, but as a second line whose purpose is to catch authoring mistakes, not to be the primary safeguard. This inversion — *containment by construction, filtering as backstop* — is the report's principal architectural claim.

2.6 Comparable games

Four titles inform the design. *Home Safety Hotline* (Night Signal Entertainment, 2024) is the acknowledged inspiration: the player answers calls describing household hazards and must consult a growing internal database to issue the correct advisory, with the entire challenge residing in interpretation and lookup rather than in dexterity. It demonstrates that a call-centre loop can carry a full game, but its knowledge base is static reference material — there is no simulated system whose state the player changes. *Papers, Please* (Pope, 2013) demonstrates the mechanic this project adapts for identity verification: cross-referencing documents against rules under time pressure, where the interesting failures come from inconsistencies the player must notice unprompted. *Hypnospace Outlaw* (Tendershoot, 2019) demonstrates a diegetic simulated operating system as the primary interface. Finally, commercial IT-support simulators such as *IT Specialist Simulator* establish market interest in the subject matter while illustrating the limitation this project targets: faults are typically discrete scripted events with scripted remedies rather than states in a coupled model.

2.7 Research gap and positioning

Synthesising the four bodies of work yields a specific gap. Serious-games research establishes that simulation can train procedural skill but says little about how to *represent* a coupled technical system so that diagnosis is genuinely inferential. Troubleshooting-instruction research (Jonassen and Hung, 2006) specifies what such an environment must contain but not how to build one as a playable artefact. Model-based diagnosis (Reiter, 1987; de Kleer and Williams, 1987) supplies the formal machinery but targets automated reasoners, not human learners. LLM-agent research supplies convincing conversational partners but has not addressed adversarial information containment where leaking the answer destroys the artefact's purpose.

This project sits at the intersection: a playable troubleshooting environment whose fault model is state-based and dependency-propagated in the model-based-diagnosis tradition, whose difficulty progression is organised by *discriminability* between confusable faults, and whose conversational agent is contained by architecture rather than by filtering.

3. Methodology

3.1 Research paradigm: Design Science

The project is conducted as Design Science Research (DSR), which is the appropriate paradigm when knowledge is produced by building and evaluating an artefact rather than by testing a hypothesis about a naturally occurring phenomenon (Hevner *et al.*, 2004). Hevner *et al.* require an artefact that addresses a relevant problem, is rigorously evaluated, and contributes design knowledge beyond the instance itself. Peffers *et al.* (2007) operationalise this as six activities, mapped here as follows:

DSR activity (Peffers <i>et al.</i> , 2007)	Realisation in this project
Problem identification and motivation	§1.1–1.2: troubleshooting skill is not trainable at a live service desk
Objectives of a solution	§1.3 objectives O1–O6
Design and development	Specification (Ch. 4) → web prototype → Unity implementation (Ch. 5)
Demonstration	Playable artefact; cascade smoke test over all forty faults (§6.2)
Evaluation	Coverage and discriminability analysis; containment analysis (§6.3–6.5)
Communication	This report and the four-document design specification

DSR also frames the report's honest limitation. Hevner *et al.* (2004) distinguish *design* evaluation — does the artefact work as specified — from *utility* evaluation against the problem environment. This project completes the former and does not attempt the latter, since an untested learning claim presented as a measured one would be a more serious defect than an absent study.

3.2 Specification-first development

An unusual methodological choice was made deliberately and is worth defending, because it shaped everything downstream: the design was written as a formal specification *before* any implementation, and the specification remained the authoritative document throughout. It comprises four linked Markdown documents totalling 1,465 lines:

Document	Lines	Content
POS_TechSupport_GameDesign.md	428	Core loop, seven invariant principles, four-layer architecture, resolution semantics, fault corpus, milestones, model selection
app.md	197	Desktop applications, module-to-application mapping, dependency cascade, transaction model, caller authorisation
schema.md	313	Data-asset schemas, runtime classes, enumerations, data-level dependency map
manager.md	527	Per-service schema, mechanism, purpose and call order

Two properties of this specification are methodologically significant. First, it separates *invariants* from *decisions*. Seven numbered principles (§4.2) are declared non-negotiable, and every subsequent decision is justified against them. This gives the project a stable evaluation criterion: a design change can be assessed by asking which invariant it violates, rather than by appeal to taste. Second, it documents *rejected* alternatives inline. The specification records, for example, why guidance-article matching is keyed on issue identifier

rather than category (category matching would make behaviour depend on array order, so a five-day trainee could be handed the wrong article after an unrelated content edit), and why only fault *field names* rather than fault *values* are checked by the output guard ("Empty" is an ordinary English word about a paper tray, and banning it would gag honest speech). Recording the reasoning, not just the outcome, is what makes the document survive as a design contract rather than decaying into stale documentation.

3.3 Prototype-then-port

Implementation proceeded in two stages rather than directly to the target platform. A complete browser-based prototype (1,969 lines of JavaScript plus HTML and CSS, approximately 2,700 lines total) was built first, then ported to Unity C#. This follows the playcentric prototyping tradition (Fullerton, 2014), in which the cheapest possible artefact that can be played is built before the expensive one, and it produced three specific benefits:

1. **Rule validation before engine cost.** The cascade rules, resolution semantics and verification flow were exercised in a medium with instant iteration, so the Unity port could be a *translation* of validated behaviour rather than a simultaneous design-and-build exercise.
2. **An executable reference.** During porting, the prototype served as an oracle: the C# `DependencyGraph` is documented as a 1:1 port of the prototype's `effectiveStatus`, `staffLoginStatus`, `dbConnected`, `runTest` and `checkState` functions, so a behavioural disagreement between the two indicated a porting error rather than an open design question.
3. **Discovery of structural problems.** The prototype's ticket construction combined two sources of fault selection — random selection from a day-appropriate pool and forced selection from a developer picker — behind one optional parameter. The port recognised this as a Factory Method situation and separated the sources (§5.4), a refactoring that later admitted a third source (recurrence) without modifying either existing one.

The cost of this approach is duplicated implementation effort. The judgement made — and, with hindsight, vindicated — was that discovering the Blocked-versus-Error distinction (§4.6) in JavaScript was substantially cheaper than discovering it in a partially built Unity scene.

3.4 Milestone-driven build order

The specification prescribes seven milestones, each with an explicit done-criterion, and forbids working out of order:

Milestone	Scope	Done-criterion	Status
M1	Data schemas, module state, dependency graph	Inject a fault; correct cascade appears in logs	Complete
M2	Desktop actions, resolution checking, minimal remote UI	One printer ticket playable end to end	Complete
M3	Shift clock, ticket queue, scoring, end-of-night	A full eight-minute shift with multiple tickets	Complete
M4	Customer AI: ground truth, intent, policy, guard	Natural chat that stays non-technical and never leaks	Complete
M5	Verification, mailbox, SMS	CRM verification and complaint strikes functioning	Complete

Milestone	Scope	Done-criterion	Status
M6	Campaign, consequence, save/load, win-lose	Sixty-day campaign persists and terminates correctly	Complete
M7	Voice (speech-to-text / text-to-speech)	—	Descoped

The value of an ordering constraint of this kind is that each milestone's done-criterion is a falsifiable statement about the artefact, so progress is measured by demonstrable capability rather than by lines written. It also enforced the layering: because M1 had to satisfy its criterion using only the Simulation layer, that layer necessarily has no dependency on managers or UI, which is why the smoke test in §6.2 can exercise the fault model on a project whose content assets have never been generated.

Notably, M4 was initially descoped and later implemented. The intermediate state — a customer represented by a fixed menu of canned lines — is itself methodologically useful evidence: because the dialogue interface was designed as a boundary from the outset, replacing the placeholder with the full four-stage pipeline required no changes to the ticket, verification or resolution systems.

3.5 Tools and environment

Concern	Choice	Justification
Engine	Unity 6 (6000.5.4f1), URP 17.5.0	Mature 2D/UI tooling; <code>ScriptableObject</code> asset model central to the architecture
UI	Unity UI (uGUI) 2.5.0, built at editor time	Persistent, inspectable GameObjects rather than runtime-generated hierarchy
Input	Input System 1.19.0	Current supported input stack
On-device inference	Sentis (<code>com.unity.ai.inference</code> 2.6.1)	Installed for a future intent-classification model (specification option A)
Language model	Ollama serving a small instruct model (default <code>llama3.2:3b</code>)	Local self-hosting: no cloud dependency, no API cost, no player data leaving the machine
Version control / docs	Markdown specification alongside source	Specification travels with the code it governs

The language-model decision deserves comment because it was constrained by hardware. The development machine is an Apple M1 Pro with 16 GB of unified memory, of which the Unity editor consumes 4–8 GB. This rules out models above roughly the 14-billion-parameter class and makes a ~3B instruct model in 4-bit quantisation (≈ 2 GB) the appropriate target. The architecture converts this constraint into an advantage: because the model's only job is to reword a line the policy has already decided, a small model is not merely acceptable but sufficient, and the game remains fully playable with no model installed at all.

3.6 Evaluation approach

Four evaluation instruments were applied, and their limitations are stated with them:

1. **Cascade verification (§6.2).** A harness injects each of the forty authored faults into a fresh simulated desktop and prints the resulting status of all six modules. The expected pattern is asserted by inspection: machine-wide faults must show `Blocked` down the whole chain, and every other fault must show

Error on exactly the module that owns the symptom and **OK** elsewhere. *Limitation*: assertion is by human reading of log output, not by automated test.

2. **Specification coverage audit (§6.3)**. Each specified mechanism is traced to implementing code or recorded as unimplemented. *Limitation*: establishes presence, not correctness.
3. **Discriminability audit (§6.4)**. Each authored fault is checked against its declared confusable neighbour to confirm that observable evidence distinguishes them. *Limitation*: the author is also the assessor.
4. **Containment analysis (§6.5)**. The information available to the dialogue agent is traced against the information required to state the root cause. *Limitation*: an argument from construction, not a red-team exercise; §8.2 proposes the adversarial study that would be required.

No instrument here measures learning. Establishing that would require a controlled study of the kind reviewed by Wouters *et al.* (2013), with pre/post diagnostic assessment and a delayed retention measure; §8.2 specifies its design.

4. Design

4.1 The core loop

The player is a probationary technical-support agent on the night shift, 20:00 to 04:00, compressed into eight minutes of real time. The campaign is sixty nights. Each night, calls arrive at a configured rate on a tempo curve — sparse and easy early in the campaign, dense and compound late — and each call is a ticket proceeding through six phases:

1. **Answer.** An incoming-call popup rings for twelve seconds; letting it lapse counts as a missed call and files a complaint.
2. **Verify.** The caller states a store name, an owner name and a register identifier. Any of these may be wrong, because personas have a memory-accuracy trait. The player searches a CRM, selects among results that include decoys, and cross-checks the caller's claims against the record.
3. **Connect.** Remote access requires the correct remote identifier for the *selected* CRM record plus a per-session passcode. Selecting the wrong record does not raise an error — the connection simply fails, because verifying is the player's job, not the system's.
4. **Diagnose.** Inside a simulated desktop of seven applications, the player runs diagnostic actions that reveal clues about module state. Faults must be traced against the dependency graph.
5. **Repair.** Fix actions write module state. Some are gated by preconditions; some are marked risky and can make matters worse.
6. **Close.** The ticket receives a health verdict — Resolved, Degraded, or still In Progress — recomputed from current state at the moment of closing.

Failure is graduated rather than binary. Three complaint emails fail a night; a failed night adds a warning; exceeding the warning threshold ends the campaign. Winning requires surviving all sixty nights *and* resolving at least 150 tickets, so neither caution nor speed alone suffices — an application of the calibrated-challenge principle (Malone, 1981; Csikszentmihalyi, 1990).

4.2 Seven invariant design principles

The specification declares seven principles as inviolable. They function as the project's architectural constitution: each is a constraint that, if relaxed, would silently destroy something the design depends upon.

P1 — The issue asset is the single source of truth about a fault. Every system reads the fault definition from one authored asset. This prevents the classic content bug in which a symptom, a clue and a repair condition drift apart across three files.

P2 — The customer never knows the root cause. The conversational agent is supplied only with lay symptom descriptions and identity information. Technical symptom text, fault definitions, clues and resolution conditions are structurally absent from its input. This is the containment claim of §2.5, elevated to an invariant.

P3 — A fault is invalid state in a module, not a boolean flag. Repair means bringing state back to a valid value. This is what allows unauthored fault combinations to behave coherently, in the model-based-diagnosis tradition (Reiter, 1987).

P4 — Components are linked; faults propagate downstream, repairs proceed upstream. Symptoms surface at the downstream end of a dependency chain; the fix is at the upstream end. This is the device-knowledge requirement of Jonassen and Hung (2006) rendered as a mechanic.

P5 — The customer is non-technical. They misdescribe, misname devices, and misattribute causes. The player must verify rather than believe. This makes the human report a *noisy sensor*, which is what real support work involves.

P6 — Authored assets hold static data only; runtime state lives in plain classes. Runtime state is never written back into an asset. In Unity this is not a stylistic preference but a correctness requirement, since asset mutations persist across play sessions in the editor and would corrupt subsequent runs.

P7 — The policy is the brain; the language model is only the mouth. The model expresses what the policy has decided. It never decides what to say about the fault.

4.3 Four-layer architecture

```

LAYER 1 – DATA (authored, ScriptableObject assets, read-only at runtime)
  IssueSO · StoreProfileSO · PersonaProfileSO · DesktopActionSO
  KnowledgeArticleSO · ReceiptTemplateSO · GameConfigSO · ContentDatabaseSO
    | cloned, never mutated
LAYER 2 – SIMULATION (runtime state)
  VirtualDesktopInstance = a set of Modules holding mutable state
  Modules: OS · Network · POSSoftware · Terminal · Printer · CashDrawer
  DependencyGraph – the single owner of the Blocked/Error cascade
    | read by
LAYER 3 – LOGIC (game rules)
  ProblemGenerator · ResolutionChecker (pure) · TransactionModel · RuntimeState
    | reduced view only
LAYER 4 – AI (the customer)
  GroundTruth (the boundary) → IntentClassifier → DialoguePolicy
  → ILLmClient (optional phrasing) → GroundingGuard (final gate)

```

The critical property is the *narrowing* of information as one moves down the diagram. Layer 4 does not receive the objects of Layers 1–3; it receives a purpose-built reduced view (GroundTruth) constructed by a single factory method. This is Martin's (2017) dependency rule applied to information rather than to compilation: the AI layer cannot leak what it cannot reference. Invariant P2 is therefore enforced by the type system rather than by developer discipline — an important distinction, because discipline degrades under deadline pressure and type errors do not.

4.4 The POS domain model

The simulated ecosystem mirrors a real single-lane installation:

```

Windows/OS → Network → POS Software (hub) → Terminal (card payments)
                                   |
                                   → Database (transaction history)
                                   |
                                   → Printer → Cash Drawer

```

Seven desktop applications expose these six modules, and the mapping is deliberately *not* one-to-one:

Application	Module	Sub-tabs
System (Windows)	OS	health, services
POS Manager	POSSoftware	receipt, connections, staff, database
Printer & Print Queue	Printer	queue
Device Manager	Printer	printer
Network Settings	Network	adapter
Cash Drawer Config	CashDrawer	port
POS Terminal	Terminal	status, batch

Two applications — Printer & Print Queue and Device Manager — are views onto the *same* module. This is not redundancy; it is a designed misconception trap. A player who sees a print queue jammed in one window and a device error in another naturally infers two problems, when both are consequences of one fault. Chi, Feltovich and Glaser (1981) predict exactly this novice behaviour: categorisation by surface feature rather than by underlying principle.

The transaction model contributes a second layer of realism that generates its own faults. A transaction exists in two places at once: in a *batch* on the terminal (financial data awaiting settlement) and in *transaction history* in the POS database (an archival record, which survives batch closure). Void is legal only while a transaction is open; refund remains legal after settlement; reprint reads the archived snapshot and therefore requires a working database connection. Each rule is the basis of an authored fault, and together they encode the incident-versus-problem distinction of ITIL 4 (AXELOS, 2019) at the level of individual mechanics.

A third mechanism, **caller authorisation**, adds a non-technical failure mode. The caller is not necessarily the owner on the CRM record. Roughly forty per cent of tickets are refund-or-void cases; within those, a coin flip determines whether the staff member calling has actually been authorised by the owner, and this ground truth is fixed for the ticket so the customer answers consistently every time they are asked — a real person does not change their story between asks. If the player performs a refund or void without having established authorisation, and the caller turns out to be unauthorised, the ticket is capped at Degraded *regardless of how cleanly every technical fault was repaired*. Crucially, the penalty attaches to *harm*, not to *procedure*: a player who guesses and happens to be right is not punished. This is a deliberate design position — it rewards verification because verification prevents harm, not because a checklist demanded it.

4.5 Fault taxonomy: forty faults and the discriminability rule

Forty faults (P1–P40) are authored across seven categories. The organising rule is stated in the specification and is the single most important content decision in the project:

Each fault must be separable from a fault the player already knows. A fault that produces symptoms the player has already learned to read is not content — it is padding.

Every fault therefore carries an explicit record of *which prior fault it is confusable with* and *what evidence distinguishes them*. Selected examples:

Fault	Faulty state	Confusable with	Discriminating evidence
P1	<code>Printer.paperLevel = Empty</code>	— (baseline)	Queue reports "out of paper"
P3	<code>CashDrawer.port = COM3</code> (clashes with printer)	P2 driver corruption	Driver reports healthy; port configuration collides
P5	<code>POSSoftware.receiptTemplate = Broken</code>	Any printer fault	Test page prints correctly but the customer copy lacks fields
P13	<code>OS.spoolerService = Stopped</code>	P1, P2	Paper present, driver healthy, jobs stuck at "Spooling" — reinstalling a driver cannot restart a stopped service
P16	<code>OS.systemTime = Skewed</code>	P6, P7, P37	Wi-Fi correct, IP registration correct, cash works, every card declined — card authorisation runs over TLS, so a wrong clock breaks the handshake
P17	<code>Printer.paperJam = Jammed</code>	P1	Paper is present, plus mechanical noise
P18	<code>Printer.cableConnected = false</code>	P2	Device Manager does not list the device — absence differs from an error, and there is nothing to reinstall
P21	<code>POSSoftware.printerVisible = false</code>	P20	Two independent registrations: Windows having a printer is not POS having a printer, so a passing test page proves nothing
P23	<code>Printer.paperWidth = 58mm</code>	P5	Missing <i>fields</i> means template; complete content that is <i>truncated</i> means wrong paper width
P35	<code>Network.signalStrength = Weak</code>	P4	Intermittency is itself a diagnosis: a dead link fails always, a weak one fails under load
P36	<code>Network.dnsServer = 8.8.8.8</code>	P12	Identical error text: P12 is a mistyped host name, P36 is a correct name nobody can resolve
P39	<code>OS.userAccount = Standard</code>	P8	Two independent permission systems: the POS role governs what may be done <i>in</i> the till, the Windows account governs whether the application runs at all

Three design consequences follow.

Corrective feedback is built into the corpus. The P1/P17/P18 triad — out of paper, jammed with paper present, cable unplugged so the device is absent entirely — is a set of *near misses* around a single naive hypothesis ("the printer is broken"). Deliberate practice requires exactly this: repeated exposure to cases that discriminate (Ericsson, Krampe and Tesch-Römer, 1993).

Some faults teach that the layer of the symptom is not the layer of the cause. P13 and P16 have their fault in the OS module but surface as errors on the Printer and Terminal respectively. This is the central lesson of the corpus, and it is precisely the device-knowledge component that Jonassen and Hung (2006) argue conventional instruction fails to build.

Red herrings are authored, not incidental. Clues carry an `isRedHerring` flag; the permission fault P10, for instance, is accompanied by a genuine low-paper warning on the same screen. Real diagnostic environments contain true-but-irrelevant information, and an environment where every visible anomaly is relevant trains a false reflex.

4.6 The dependency cascade: Blocked versus Error

The most consequential rule in the design is the distinction between two failure statuses:

- **Error** — this module is *itself* misconfigured or broken. Its clues **must remain readable**.
- **Blocked** — this module is healthy but cannot reach an upstream dependency. Its clues are **hidden** until the upstream fault is cleared, and the status carries a reason pointing upstream (for example, "Terminal: cannot operate — reason: POS not connected").

The reason string is not decoration; it is the trail the player follows upstream. But the rule that gives the distinction teeth concerns the OS module, which has *two categorically different kinds of fault*:

- **Machine-wide** (`diskSpace = Full`, `pendingReboot = true`): nothing runs. The whole chain below becomes **Blocked**.
- **Service-level** (`spoolerService = Stopped`, `systemTime = Skewed`): the chain is **not** blocked. These surface as *local Errors* on precisely the module that needed the service — a stopped spooler makes the Printer report an error, a skewed clock makes the Terminal reject cards.

The temptation to treat a stopped OS service as blocking is strong and would be wrong, because blocking hides clues, and hiding the clues for a fault whose repair lies in the OS layer leaves the player with a dead end instead of a trail. The specification records this as the rule most easily violated, having in fact been violated once during development for the terminal-identity faults P6 and P7.

The network module carries the same distinction under a different name: **down versus degraded**. Only `isOnline = false` blocks the chain. A weak signal, a wrong DNS server or a blocking firewall leaves the link alive, POS running, and — critically — the clues readable. Each degradation breaks something different downstream: weak signal and firewall surface at the Terminal, wrong DNS surfaces at the database connection.

Two failure domains sit deliberately *outside* the cascade: per-staff login (a role, a terminal assignment and a synchronisation flag, evaluated in four ordered stages that stop at the first failure, sourcing faults P8–P11) and database connectivity. Both can fail while every module in the chain is healthy. Modelling them inside the cascade would imply that one person's login failure indicates a system fault, which is the misinference the design most wants the player to stop making.

Masking is then handled by fault lifecycle. A fault injected behind a blocker is **Latent**: not observable, not repairable, not yet graded. After each repair the system re-evaluates, removes cleared blockers from every latent fault's blocking list, and promotes any fault whose list has emptied to **Active** — a new problem revealing itself. Blocker relationships are derived by rule rather than authored per fault: OS machine-wide blockers block everything including the network outage, and the network outage blocks every non-blocker fault. This is de Kleer and Williams's (1987) multiple-fault masking as a game mechanic, and it teaches the lesson that clearing a blocker is the *start* of a diagnosis: at the moment everything was **Blocked**, nothing underneath had been diagnosed at all.

4.7 Resolution semantics

Ticket evaluation is defined as a pure function of current state, never stored:

```

EvaluateIssue(desktop, issue):
  if the owning module is Blocked          → Hidden      (not yet gradeable)
  if all rootCauseFixed checks pass
    and any required test passes          → Resolved
  if any of the issue's worseningFaults
    is currently present                  → MadeWorse
  otherwise                                → Unresolved

EvaluateTicket(problem):
  if an unauthorised refund/void occurred → Degraded    (business harm)
  if any issue is Hidden                  → InProgress
  if every issue is Resolved              → Resolved
  if any issue is MadeWorse               → Degraded
  otherwise                               → InProgress

```

Three aspects are pedagogically loaded.

Symptom clearance is separated from root-cause repair. An issue defines both `symptomCleared` and `rootCauseFixed`. If the symptom is gone but the cause is not, the ticket closes, the customer is satisfied, and the fault is flagged as *recurring* — returning on a later night. This is the ITIL 4 incident/problem distinction (AXELOS, 2019) turned into a delayed consequence, and it is the only mechanism in the game that punishes a player who has apparently succeeded.

Making things worse is a first-class outcome. Faults declare `worseningFaults` — states that appear when the player applies a plausible but wrong repair. The canonical case is the printer driver fault P2, whose risky repair ("remove and re-add the printer device") removes the printer entirely when the real cause was the cash drawer occupying the same port. A second case is more subtle: for the missing-role fault P8, granting *Admin* rather than *Sale* clears the symptom perfectly while conferring refund, void and batch-closure rights the staff member should not have — a repair that works and is nonetheless harmful. Risky actions require explicit confirmation, so the player commits knowingly (Reason, 1990).

Verdicts are recomputed, never cached. The specification records the bug this rule was written to prevent: a stored verdict that goes stale produces a state where the desktop is correct but the interface reports failure. Statelessness here is a correctness property, not an aesthetic one.

4.8 Verification and the noisy human sensor

Verification is split into two independent layers that can each succeed while the other fails: *which store is this* (CRM lookup) and *who is this person* (caller role and authorisation). A player can select the right store record and still be talking to the wrong person.

The mechanic is deliberately manual. There is no automatic caller identification. The player clicks one CRM field and one chat statement of the same fact type, and the system reports match or mismatch. Nothing is verified until the player performs the comparison — an application of *Papers, Please*'s core mechanic (Pope, 2013) and of Endsley's (1995) point that situation awareness is constructed by the operator, not delivered to them.

For the comparison to be worth performing, the directory must contain records worth confusing. The decoys are therefore *generated by rule* rather than authored individually. A name table holds families of mistakeable first words — {Sunrise, Sunset, Sunnyside}, {White Horse, White Hart}, {Cornerstone, Corner House} — which are crossed with a list of trades to produce accounts in confusable clusters: the same first

word under two trades ("Sunrise Diner", "Sunrise Bakery") and a sibling first word under the same trade ("Sunset Diner"). Sixteen families are authored and six are rolled per campaign, giving roughly twenty accounts, each with its own owner, address, remote identifier and passcode. A caller is drawn from that directory per ticket, so a different shop is on the line each time; and when persona memory accuracy corrupts the stated store or owner name, the wrong value is read from a *near-miss record in the same directory* rather than from a hardcoded string. The misremembered name is therefore always a shop that genuinely exists two rows away — which is the shape of the mistake the mechanic is trying to teach the player to catch.

Which near miss is chosen turns out to matter as much as having one. Candidates are *ranked* rather than pooled: a record from the same authored family that also shares a literal word ("Corner House Barbers" against "Corner House Pharmacy") outranks a family sibling with no shared word, which outranks a record that merely happens to share a trade. An early unranked implementation reached for the softest option — a shop from an unrelated family that happened to also be a bookshop — because after several families rolled the same trade there were many such records and only one true sibling. Two fixes followed from reading the generated directory rather than the code: rank the candidates, and prevent families from reusing a trade word, which in a six-family sample raised the trade-word count from five distinct trades to twelve and made the strongest confusion the one actually presented.

The same reasoning applies below the CRM. Each shop's Wi-Fi SSIDs and record-store host name are *derived* from its store name (Corner House Bakery → CornerHouseBakery-Main, CornerHouseBakery-Guest, db.cornerhousebakery.local), so the simulated desktop belongs to the shop that called rather than to whichever shop the content author happened to name. This is what forces the fault corpus to become store-agnostic, discussed as an implementation consequence in §5.3.

The customer's statements are unreliable by design, in three graded ways. Persona memory accuracy corrupts stated identifiers, so a mismatch may be an honest error. Persona technical literacy drives a *misnaming* map that rewrites device names in the customer's speech — the terminal becomes "the card machine", the POS screen becomes "the till". And persona honesty governs the SMS receipt mechanic: asked to text a receipt for cross-checking, a less honest persona sends the wrong one — a different machine, a timestamp three days old — which, if trusted, leads to a confident diagnosis of the wrong fault.

4.9 Difficulty progression and fading guidance

Content is gated by campaign day in five tiers, which manages cognitive load (Sweller, 1988) by ensuring that a new discrimination is introduced only once the previous one has been practised:

Days	Fault pool	Discrimination being taught
1–5	P1, P17, P18	Read the observable state of one device
6–15	+ P2, P19, P22, P24	Distinguish four failure kinds inside the printer
16–30	+ P3, P6, P7, P8, P13, P20, P25–P27, P30, P33–P35, P39, P40	Cause moves away from symptom; cross-layer faults appear
31–45	+ P5, P9–P12, P15, P16, P21, P23, P28, P29, P31, P32, P36–P38, and the first blocker chains	Money-shaped faults; two faults at once
46+	Full pool, including blocker-over-fault chains	Clear the blocker, then find what was hidden underneath

Guidance fades on the *same* boundary rather than on a separate configured schedule. During days 1–5, tickets automatically attach the relevant knowledge-base article; from day 6 the attachment stops and the player must search the knowledge base themselves by category or error code. Re-using the difficulty boundary as the guidance boundary is a small decision with a real consequence: "easiest faults" and "guidance provided" fade in one step instead of drifting apart into an accidental easy mode. The knowledge base contains forty articles, one per fault, and each teaches the *discrimination* rather than merely the symptom — the driver article warns that a port conflict looks identical and that "remove and re-add" is the move most likely to make things worse. This is the "query an experienced troubleshooter" component of Jonassen and Hung's (2006) architecture, in asynchronous form.

4.10 Campaign, consequence and economy

A sixty-night campaign requires state above the level of a single night. Night-scoped state (the clock, the ticket queue, the night's complaints) resets each night; campaign-scoped state (day, cumulative tickets resolved, warnings, currency) persists, as does a consequence ledger holding pending recurrences and narrative flags. The two counters are deliberately at different levels: complaints accumulate *within* a night and three of them fail it, whereas warnings accumulate *across* nights and three of those end the campaign. Collapsing them into one counter would make a single bad night unrecoverable.

Scoring is intentionally simple — ten currency units per resolved ticket, minus fifteen per degraded ticket, floored at zero. The asymmetry states the design's ethical position plainly: causing harm costs more than the credit for fixing something. Richer scoring (correct root cause, redundant steps, time taken, temporary versus permanent repair) is specified but not implemented, and is listed in §8.2.

5. Implementation

5.1 Structure and scale

The implementation comprises 7,619 lines of C# across fifty files, organised so that folder, namespace and architectural layer coincide:

Folder	Namespace	Lines	Responsibility
Core/	P0STechSupport.Core	114	All enumerations, including <code>Status</code> (OK/Error/Blocked)
Data/	P0STechSupport.Data	710	Nine <code>ScriptableObject</code> types, shared serialisable types, <code>StoreRecord</code> , <code>StoreIdentity</code> , content registry
Simulation/	P0STechSupport.Simulation	641	<code>ModuleBase</code> , six modules, <code>VirtualDesktopInstance</code> , <code>DependencyGraph</code> , <code>WifiTable</code>
Logic/	P0STechSupport.Logic	1,088	Runtime state, <code>ResolutionChecker</code> , problem generation, CRM directory generation, transaction model
Managers/	P0STechSupport.Managers	1,029	Thirteen service classes
AI/	P0STechSupport.AI	688	The four-stage dialogue pipeline
UI/	P0STechSupport.UI	1,329	<code>GameUIController</code> , <code>UIFactory</code>
Editor/	P0STechSupport.EditorTools	1,755	Content generator, scene builder, CRM directory preview (editor-only assembly)
DevTools/	P0STechSupport.DevTools	104	Cascade smoke test
GameManager.cs	P0STechSupport	161	Composition root and night loop

Two assembly definitions enforce the editor boundary: the runtime assembly `P0STechSupport` and the editor-only `P0STechSupport.Editor`, which references the runtime assembly and is restricted to the Editor platform. This guarantees at compile time that the content generator and scene builder cannot be referenced from shipped code.

That the Editor folder is the single largest component (1,755 lines, 23% of the codebase) is worth noting as a finding rather than an anomaly. A data-driven design relocates effort from gameplay code into content authoring and tooling; the tooling then becomes a first-class part of the system. This is the predicted trade-off of asset-centric Unity architecture (Hipple, 2017) and it materialised as predicted.

5.2 Data layer, and a deliberately uncomfortable trade-off

Eight `ScriptableObject` types hold all authored content. `IssueS0` is the centre, structured in four tiers that mirror the diagnostic process: fault injections (what is wrong), symptoms (lay and technical descriptions), clues (which diagnostic action reveals what) and a resolution condition (what counts as fixed).

The significant implementation decision is that **module state is stored as string-keyed values rather than typed fields**. A fault injection is a triple of module, field name and value; a state check is a quadruple of module, field name, comparison operator and expected value. Both are plain serialisable data.

The cost is real and should be stated plainly: no compile-time protection against a mistyped field name, and no type checking on values. A typo in an authored asset produces a check that silently never passes.

The benefit is that it is the property that makes the design work. Because faults, repairs, preconditions and resolution conditions are all *data*, all forty faults and all fifty desktop actions were authored without writing a single new class or method. Had module state been typed fields, each new fault would have required code — a `switch` arm somewhere, at minimum — and the corpus would have been an accumulation of special cases rather than a table. The trade-off is therefore compile-time safety exchanged for content velocity and uniformity, and given that content volume is what determines whether the game can sustain 150 tickets, the exchange is favourable. Mitigation is the smoke test of §6.2, which exercises every authored field name against the live modules and would surface a typo as an absent cascade.

`ContentDatabaseS0` acts as a single registry aggregating every authored asset, so `GameManager` requires exactly one serialised reference in the scene rather than dozens.

5.3 Simulation layer

`ModuleBase` provides string-keyed state with typed accessors and an abstract `LocalStatus` method returning a `StatusResult` (a status plus a human-readable reason). Six concrete modules implement `LocalStatus` as the local fault rules for their own component only. No module knows about any other module — all coupling lives in one place.

`DependencyGraph` is that place. It resolves the cascade of §4.6 and is documented as the *only* code permitted to distinguish `Blocked` from `Error`. Its structure follows the dependency order directly, with the two subtleties from §4.6 visible in code:

```
case ModuleType.POSSoftware:
{
    if (OsBlocking(out string why))
        return new StatusResult(Status.Blocked, $"cannot operate - reason: {why}");
    // Blocked only when the network is DOWN. A degraded link (weak signal, bad DNS,
    // firewall) leaves POS running - and leaves its clues readable, which is the point.
    if (d.GetModule(ModuleType.Network) is NetworkModule net && net.IsDown())
        return new StatusResult(Status.Blocked, "cannot operate - reason: Network offline");
    return d.GetModule(ModuleType.POSSoftware).LocalStatus(d);
}
```

`OsBlocking` returns true only for machine-wide OS faults, so a stopped print spooler propagates no block and instead surfaces as a local Printer error — the rule of §4.6 implemented as a single predicate rather than as scattered conditionals.

The graph also owns the two out-of-cascade failure domains. `StaffLoginStatus` evaluates the four ordered permission stages and returns at the first failure, so the four permission faults P8–P11 are distinguished by *where the sequence stops* rather than by separate authored logic. `DbConnected` returns the same user-visible error text for a mistyped host and for an unresolvable correct host — the deliberate ambiguity of the P12/P36 pair, implemented as identical output from different causes.

`WifiTable` models DHCP honestly: a terminal's IP address and gateway are *derived* from the network it has joined, rather than being independently editable fields. Joining a guest network therefore moves the terminal into a different address range as a consequence, which is why fault P6 (wrong Wi-Fi) and fault P7 (stale IP registration on the POS side) are genuinely different faults with genuinely different repairs rather than two spellings of "wrong network settings".

`WifiTable` addresses those networks by *role* — the shop's own network is 192.168.1.x, the guest network 192.168.50.x — while their *names* come from a `StoreIdentity` derived from the calling shop (§4.8). The split is deliberate: how a site is wired is a fact about the installation, whereas what it is called is a fact about the business. Since a fault asset must then be able to say "the guest network" without naming a shop, authored content writes **tokens** (`{SSID}`, `{SSID_GUEST}`, `{DB_HOST}`, `{DB_HOST_TYPO}`) and the values are substituted at three chokepoints: state writes (`VirtualDesktopInstance.Apply`), state comparisons (`DependencyGraph.CheckState`, which serves both resolution conditions and action preconditions) and player-visible text (`ActionManager`'s log, the knowledge base). P6 is therefore authored once as "joined `{SSID_GUEST}`", resolved when `wifiNetwork` equals `{SSID}` and is a correct, fixable fault for every shop in the directory; P12's mistyped host is generated from the shop's own hostname by inserting a hyphen where the words join, so the near-miss that makes it hard to spot survives the generalisation instead of being a property of one authored string.

That the tokens are checked in the *comparison* layer rather than at authoring time is what keeps the invariant honest. A resolution condition of "equals `{SSID}`" cannot silently mean the wrong shop's network, because there is exactly one place where an expected value is resolved and it reads the identity off the desktop being graded.

5.4 Logic layer

`ResolutionChecker` is a static class of pure functions implementing §4.7 exactly. It holds no state, which is the mechanical guarantee behind the "recompute, never cache" rule.

`ProblemGenerator` is where the Factory Method pattern (Gamma *et al.*, 1994) does real work. The prototype combined random and forced fault selection behind one optional parameter; the port separates *choosing which faults* from *assembling the ticket*:

```
public interface IProblemFactory { ProblemInstance Create(int day); }

class RandomPoolProblemFactory : IProblemFactory // day-appropriate pool, randomised
class ForcedIssueProblemFactory : IProblemFactory // developer picker, exact combination
class RecurringProblemFactory : IProblemFactory // decorator: prefers a due recurrence
```

All three delegate assembly to a shared `ProblemAssembler`, which itself composes two sub-factories: `DesktopFactory` (clone the healthy baseline, apply each fault) and `PersonaFactory` (roll the refund/void case, derive caller role, authorisation and possibly-mistaken stated facts). Guidance lookup enters through an `IGuidanceSource` interface, so the assembler depends on an abstraction rather than on the knowledge-base service — preserving the layering rule that Logic must not depend on Managers.

The payoff was verified in practice. When cross-night recurrence was added, `RecurringProblemFactory` was introduced as a *decorator* around the existing auto factory, activated by a single call at composition time:

```
Generator.EnableRecurring(Consequence.DueRecurringToday, Consequence.ConsumeRecurring);
```

Neither existing factory was modified. This is the open/closed principle producing a measurable result rather than a stylistic preference.

Day pools are authored data (`IssuePool[]`) rather than code, with one implementation detail worth recording: combinations are wrapped in an `IssueCombo` class because Unity cannot serialise jagged arrays.

How busy a night is began as the hardcoded `clamp(round(2 + 0.05 × day), 1, 6)` and is now expressed the way the fiction states it: a **rate**, `callsPerHour` on `GameConfigS0`, multiplied by the eight in-game hours of the shift and clamped to a floor and a safety ceiling. A per-day ramp term reproduces the original curve exactly (0.25 calls per in-game hour plus 0.00625 per day gives two calls on night one and five on night sixty), so the tuning that the prototype validated is preserved as a default rather than as a constant. The hardcoded formula survives as the fallback when no config asset is wired, which keeps `ShiftManager` testable without one. The knob is also editable live from the hub screen, and the field it writes is the same `callsPerHour` the inspector shows — one number, two surfaces, which is the same rule the guidance boundary follows in §4.9.

5.5 Services and the composition root

The specification describes sixteen managers as `MonoBehaviour` components. The implementation deviates: fifteen are plain C# classes constructed and owned by a single `GameManager` `MonoBehaviour`, and `ResolutionChecker` is static.

The deviation is deliberate and, on reflection, an improvement. Plain classes have explicit constructor dependencies, are instantiable in a test without a scene, and cannot be accidentally duplicated or misconfigured in the inspector. `GameManager` supplies what genuinely requires Unity — the lifecycle, the frame tick, and the campaign→shift→campaign flow — and exposes events (`IncomingCall`, `NightEnded`, `GameFinished`) so the UI observes rather than polls. Construction order in `BuildServices` encodes real dependencies; the knowledge-base service is built before the generator because it is the generator's guidance source.

The night loop is a clean chain of single responsibilities: `CampaignManager.StartNight` → `ShiftManager.BeginShift` → per-frame `Tick` advancing the clock and spawning calls → `TicketManager` managing lifecycle → `ResolutionChecker` computing verdicts → `MailboxManager` filing complaints → `ScoreManager` computing the night's score → `ConsequenceManager.Commit` → `CampaignManager.OnNightEnded` → `SaveManager.Persist`. Each service has exactly one privilege: only `MailboxManager` creates complaints, only `ScoreManager` computes currency, only `SaveManager` touches storage. Single-writer discipline of this kind is what prevents the class of bug where two systems increment the same counter.

5.6 The AI layer: containment in code

The dialogue pipeline is the implementation's most architecturally interesting component. It runs in four stages.

Stage 0 — the boundary. `GroundTruth` is the only view of a ticket the AI layer ever receives. It carries the caller's name and role, the persona profile, the three possibly-mistaken stated facts, the authorisation ground truth, and a list of lay symptom strings. Its construction shows the containment mechanism precisely:

```
foreach (var s in issue.symptoms)
    if (!string.IsNullOrEmpty(s.layman))
        g.visibleSymptoms.Add(s.layman);    // .layman only – .technical stays behind
```

Absent, and documented as required to stay absent: `IssueS0`, `ActiveFault`, `Symptom.technical`, `DiagnosticClue`, `ResolutionCondition`, `VirtualDesktopInstance`. Invariant P2 is thus a property of the type rather than of the prompt.

Stage 1 — intent. `IntentClassifier` maps player text to one of fourteen intents using deterministic keyword matching. This is a considered choice, not a shortcut: rules cost nothing, need no download, behave identically every run, and are unit-testable. Order matters — jargon detection runs *first*, so "did you check the printer driver" classifies as `AskTechnical` rather than matching the friendlier "printer" rule beneath it. Because the classifier is an interface-shaped component, a Sentis or language-model classifier can replace it without the three stages below noticing.

Stage 2 — the policy. `DialoguePolicy` decides what may be said, returning a `DialogueAct` (one of thirteen kinds, a content string, an optional fact reference for click-to-compare, and an end-call flag). Every trick in the game lives here. The `KnowledgeBoundary` is a hard stop placed before all other handling:

```
if (intent == PlayerIntent.AskTechnical)
{
    state.patience -= 0.15f;
    return new DialogueAct { kind = DialogueActKind.DeflectTechnical, content =
    DeflectLine(truth, state) };
}
```

No phrasing of a technical question, and no number of repetitions, can produce a technical answer — the reply is deflection, and repetition costs patience. Two further details matter. Repeat symptom requests return the *same* observation reworded, never a new fact, because a person who can only see one thing cannot report two. And the authorisation answer is read from fixed ticket ground truth, so asking twice yields the same answer; an unauthorised caller admits it and hangs up, closing the ticket in a neutral state — no strike, because refusing to proceed with an unverified caller is correct behaviour, but no resolved credit either, because no technical problem was solved.

Stage 3 — phrasing, and the ordering that makes it safe. `ILlmClient` has two implementations: `TemplateLlmClient` (the default, `Enabled = false`, a no-op because the policy's own phrasing is already in character) and `OllamaLlmClient` (opt-in, posting to a local Ollama endpoint). The ordering is the crucial design decision, and it is inverted from the obvious one:

*The template line is posted to the chat **first**. If the model is enabled and replies in time, the same `ChatLine` object is overwritten in place.*

Enabling the model can therefore only make wording more natural; it can never stall a call, and removing the model leaves the game fully playable. The client also enforces its own sanity bound — a reply longer than 240 characters is discarded on the grounds that a model which rambles has misunderstood a rewording task — and a four-second timeout with graceful fallback.

Stage 4 — the guard. `GroundingGuard` inspects every candidate line before display, checking two things. First, banned jargon, using the *same* vocabulary list as the classifier so that "the agent cannot ask it" and "the customer cannot say it" can never drift apart. Second, leakage of a fault's state *field name*. Only field names are checked, never fault values, and the reasoning is recorded in the source: "'Empty' is a perfectly ordinary word for a customer to use about a paper tray, and banning it would gag honest speech." A failed check is not surfaced as an error; the line silently falls back to the policy's template, which is safe by construction. The guard also runs on template lines — if a template fails, that is an authoring bug worth catching before it ships rather than in a player's session.

Notably, the guard sits *outside* the AI boundary and is therefore allowed to see the full problem instance. This is a coherent asymmetry: the component that must not leak the answer does not have it, and the component that checks for leaks must.

`CommunicationManager` retains only genuinely channel-level responsibility: mapping quick-ask buttons to intents so that buttons and free text enter the *same* pipeline (preventing a customer who is polite in chat and curt over SMS), and the SMS receipt mechanic, whose correctness is a persona honesty roll rather than a dialogue decision.

Against the mitigation taxonomy of §2.5, this pipeline addresses all three leak mechanisms structurally: hallucination cannot invent the root cause because the model receives no root cause and its output is bounded to a rewording of a fixed line; sycophancy cannot concede a technical claim because the policy intercepts technical intents before any model is consulted; and prompt injection has nothing to extract, because the sensitive material is not in the context window. Guardrails in the sense of Rebedea *et al.* (2023) are present, but as the second line rather than the first.

5.7 User interface

`GameUIController` (1,158 lines) binds the entire interface. The notable implementation decision is `GameSceneBuilder`, an editor menu command that constructs the full UI hierarchy as *persistent*, *inspectable* *GameObjects* and wires every `GameManager` and `GameUIController` reference, rather than generating the hierarchy at runtime. It is re-runnable, removing any previous canvas, game system and event system first.

This gives a designer-editable scene with programmatic reproducibility — the layout can be hand-adjusted afterwards, but a broken scene can be regenerated from source in one command. For a project whose UI comprises seven simulated applications with sub-tabs, five screens, and seven overlays, hand-building the hierarchy would have been both slow and fragile.

Interface behaviour follows a rule established early and recorded as project feedback: **tools are always present; only their results depend on the ticket.** Every application, every diagnostic action and every input field renders unconditionally, and only outcomes vary with ticket state. Gating whole interface sections on ticket type railroads the player and creates dead ends — an early prototype hid the IP-entry section on

Wi-Fi tickets, removing the very control needed to investigate. Since the design objective is self-directed diagnosis, the interface must be a consistent sandbox. Open sub-tabs are remembered per application across close and reopen, and clue revelation is scoped to the open tab, so exploration is rewarded at a fine grain.

Three interface details are worth recording because each fixes a way the layout fought the player rather than the fault. Every growing list — the call log, the chat transcript, the CRM results, an application body, the knowledge base, the mailbox — is built inside a scroll view rather than a fixed panel, so content that outgrows its box scrolls instead of spilling; the chat additionally snaps to the newest line only when a line is actually added, so scrolling back through a call survives the per-frame refresh. The developer force-call strip is hidden behind a toggle, since a debug tool permanently occupying the night screen is not part of the game. And that strip is laid out as a wrapping grid: twenty buttons in a horizontal layout group are squeezed to a few pixels each and every label breaks one character per line, which is the kind of defect that is invisible in code and obvious in a screenshot.

Art direction uses a retro Windows aesthetic (nine-sliced window frames, headers, buttons) to make the simulated desktop legible as a desktop without the cost of bespoke art.

5.8 Content authoring pipeline

`SampleContentBootstrap` (a large part of the 1,755-line editor assembly) generates the entire content corpus as assets under `Assets/Content/Generated` from a single menu command: forty issues, forty knowledge-base articles, fifty desktop actions, a store-name table, one template store profile, one persona, receipt templates, a game configuration and the content database.

Authoring content in code rather than by hand in the inspector is an unusual choice with three concrete justifications. Forty faults each with faults, symptoms, clues and resolution conditions is thousands of inspector fields, and hand-entry at that volume produces silent typos. The corpus is also *cross-referential* — blocker relationships, guidance mappings, action-to-clue links — so a generator can wire relationships **by rule** rather than by hand. `WireBlockers` is the clearest example: rather than authoring each fault's blocking list, it applies the rule that OS machine-wide blockers block everything including the network outage and the network outage blocks every non-blocker. Before this existed, the blocking field was always empty in practice, which meant the entire `Latent → Active` promotion branch had never once executed — a mechanic fully implemented and completely unreachable. Finally, regenerating is idempotent, so content evolves with the code that consumes it.

The CRM directory extends this from *wiring* to *content itself*, and the defect it fixes is worth recording because it was invisible from the code. Every specified mechanism around verification was implemented and reachable, yet one hand-authored store profile meant the same shop called on all sixty nights and the directory held two hand-written decoys. The mechanic worked; the content made it trivial. Replacing the three authored records with a word table plus a generation rule (§4.8) is a change of about two hundred lines that restores the discipline the design asks for, and it generalises the discriminability rule of §4.5 out of the fault corpus and into identity: the near-miss is no longer a string an author remembered to write, it is a structural property of how the directory is built. The lesson is the one the report keeps arriving at from different directions — in a data-driven design, a mechanic is only as strong as the content authored against it, and "implemented" is not the same claim as "exercised".

6. Testing and Evaluation

6.1 Verification strategy and its honest limits

The verification strategy has three components, and its principal weakness should be stated before its results: **the project contains no automated unit or integration test suite**. The Unity Test Framework (1.7.0) is installed, but no test assembly definition exists and no tests are authored. Verification rests on a purpose-built cascade harness, structured manual playtesting in the editor, and the earlier browser prototype acting as a behavioural oracle during porting.

For an artefact of this size this is a genuine deficiency rather than a defensible economy, and it is the first item of future work in §8.2. Its mitigating circumstance is architectural: because the Simulation and Logic layers are plain C# with explicit dependencies and no Unity coupling, `DependencyGraph`, `ResolutionChecker` and all three problem factories are directly instantiable in tests. The architecture is test-ready; the tests were not written.

6.2 Cascade verification over all forty faults

`SimulationSmokeTest` is the M1 done-criterion made executable. It builds a fresh simulated desktop, injects one authored fault, and prints the effective status and reason of all six modules — repeated for each of the forty faults, plus a healthy baseline, giving forty-one cascade readings. It requires no content assets at all, driving the Simulation layer directly, so it remains valid on a project where the generator has never been run. The two store-named faults (P6 wrong Wi-Fi, P12 mistyped record host) are injected as tokens and resolved against the generic site identity, so the harness stays store-agnostic in exactly the way the corpus now is (§5.3).

The expected pattern is explicit and is the property most worth protecting:

- A machine-wide fault (network offline P4, disk full P14, pending reboot P15) must show `Blocked` all the way down the chain.
- **Every other fault must show `Error` on exactly the module that owns the symptom, and `OK` elsewhere.**

The second condition is the valuable one. An *unexpected* `Blocked` is the bug this harness exists to catch, because it means a clue the player needs has been hidden behind a dependency — the failure mode of §4.6, which occurred once in development. The harness converts a subtle design violation into a visible, grep-pable difference in log output.

Observed results match the expected pattern for all forty faults. Two results are worth reporting specifically because they confirm the design's least intuitive rules:

- **P13 (spooler stopped) and P16 (clock skew)** have their fault in the OS module and yet produce `Error` on `Printer` and `Terminal` respectively, with the rest of the chain `OK`. The service-level/machine-wide distinction is therefore live in the implementation, not merely in the specification.
- **P35, P36 and P37 (weak signal, wrong DNS, blocking firewall)** produce `Error` on `Network` while leaving `P05Software` and below operational. Down-versus-degraded is likewise live.

Two limitations apply. Assertion is by human reading rather than by automated comparison against expected values, so the harness detects regressions only if someone reads the log. And it injects *single* faults, so multi-fault masking and the Latent → Active promotion path are exercised only in play, not in the harness. Both gaps are cheap to close and are specified in §8.2.

6.3 Specification coverage audit

Tracing every specified mechanism to implementing code yields the following:

Specified mechanism	Status	Evidence
Eight authored asset schemas (plus the store-name table)	Implemented	Data/ , 710 lines
Six simulated modules with local status	Implemented	Simulation/Modules/Modules.cs
Blocked/Error cascade	Implemented	DependencyGraph.EffectiveStatus
Machine-wide vs service-level OS faults	Implemented	OsBlocking predicate
Down vs degraded network	Implemented	NetworkModule.IsDown
Four-stage staff login	Implemented	StaffLoginStatus
Database connectivity as independent check	Implemented	DbConnected
DHCP-derived terminal addressing	Implemented	WifiTable
Latent → Active promotion	Implemented, now reachable	DesktopManager.OnFixApplied + WireBlockers
Resolution semantics incl. MadeWorse	Implemented	ResolutionChecker
Three problem factories + assembler	Implemented	Logic/ProblemFactories.cs
Five-tier day-gated content pools	Implemented	IssuePool.DefaultTable
Shift clock, tempo, ring timeout	Implemented	ShiftManager
Ticket lifecycle incl. end-of-shift flush	Implemented	TicketManager
CRM lookup, click-to-compare, remote connect	Implemented	VerificationManager
Generated CRM directory in confusable clusters	Implemented	StoreNameTableS0 , StoreDirectoryFactory
Store-derived site identity (SSID, record host)	Implemented	StoreIdentity , resolved in Apply / CheckState
Caller authorisation incl. unauthorised cap	Implemented	AuthorizationState , EvaluateTicket
Transaction/batch model incl. reprint	Implemented	TransactionManager
Four-stage dialogue pipeline	Implemented	AI/ , 688 lines
Optional local language model	Implemented, off by default	OllamaLlmClient
Mailbox strikes, night failure	Implemented	MailboxManager
Sixty-night campaign, win/lose, save	Implemented	CampaignManager , SaveManager

Specified mechanism	Status	Evidence
Forty faults, forty articles, fifty actions	Implemented in generator	SampleContentBootstrap
Cross-night recurrence	Implemented but dormant	See §6.6
Detailed scoring (root cause, steps, time)	Not implemented	Linear formula only
Voice (M7)	Not implemented	Descoped
Trust trait affecting persona tone	Not implemented	Field exists, unused

Twenty-four of twenty-eight specified mechanisms are implemented and reachable; one is implemented but dormant; three are not implemented, two of which were explicitly descoped in advance. A separate discrepancy is recorded for completeness. The asset folder currently holds the full corpus (forty issues, fifty actions, forty articles, all wired into the content database), but it was generated *before* store names were made data-driven, so three files still carry hardcoded values: the template store's baseline names one shop's Wi-Fi and record host, and the P6 / P12 assets hold that shop's SSID and hostname as literals rather than as tokens. Until the generator is re-run, every rolled shop would come up on the template shop's network and P6 would be unfixable, because the SSID its resolution condition demands is not one the terminal is offered. The generator is the source of truth and one menu command reconciles it; recording the failure mode matters more than the fix, because it is the exact class of defect a data-driven design invites — code and content versioned separately, with only the content stale.

6.4 Discriminability audit

Objective O2 requires that each fault be separable from a fault the player already knows. Auditing all forty against their declared confusable neighbours, every fault has at least one *observable* discriminator reachable through a diagnostic action — not merely a conceptual difference. Three patterns emerge:

1. **Presence versus absence** (P1 vs P17 vs P18): paper absent, paper present with mechanical noise, device not listed at all. Discriminated by direct observation.
2. **Layer separation** (P5 vs printer faults; P21 vs P20; P39 vs P8): the same visible complaint originates in different registration or permission layers. Discriminated by a test that isolates layers — the test page needs no transaction data, so it passes when the fault is in the POS template, which is *itself* the discriminator.
3. **Identical error text, different cause** (P12 vs P36; P16 vs P37): the discriminator is not in the error message but in a second observation elsewhere. The knowledge base explicitly teaches the ordering ("check the clock first — one second and half the problem is eliminated").

The third pattern is the strongest evidence that the corpus trains inference rather than recall, since no lookup from symptom to cause can resolve it. Its weakness as evaluation is equally clear: the assessor is the author, and expert intuition about what is discriminable is exactly what a novice lacks. Confirming that these discriminations are learnable *by players* requires the study in §8.2.

6.5 Containment evaluation of the dialogue agent

Objective O4 concerns whether the language model can leak the answer. The specification claims a five-layer enforcement of non-technicality; tracing each layer to code:

Layer	Mechanism	Implementation	Assessment
1. Data	Only lay symptoms cross the boundary	<code>GroundTruth.From</code> copies <code>.layman</code> only	Structural. Type-enforced; the strongest layer
2. Persona	Technical literacy capped at 0.7	<code>PersonaProfileSO</code> range	Content-dependent; an authored persona could violate it
3. Policy	Technical intents deflected before all other handling	<code>DialoguePolicy.Decide</code> first branch	Structural for classified intents
4. Generation	Misnaming applied; template phrasing default	<code>GroundTruth.Misname</code> , <code>TemplateLlmClient</code>	Effective; misnaming is probabilistic by design
5. Guard	Jargon and state-field-name filtering	<code>GroundingGuard.IsSafe</code>	Backstop; shared vocabulary prevents drift

The load-bearing observation is that layers 1 and 3 are *structural* while layers 2, 4 and 5 are *behavioural*. Even if every behavioural layer failed simultaneously — a maximally leaky persona, a jailbroken model, a guard with an incomplete word list — the model still could not state the root cause, because the root cause was never in its context. This is the inversion argued for in §2.5, and it is the property that makes a small, locally hosted, non-safety-tuned model acceptable in a role where information leakage would be fatal to the artefact's purpose.

Two residual risks are identified honestly. First, the intent classifier is a keyword matcher, so a technical question phrased without any listed keyword may fall through to `Unknown` — which yields a confused reply, a safe failure, but a less convincing one. Second, the jargon list (approximately fifty terms) is necessarily incomplete; its incompleteness degrades *plausibility* rather than *containment*, which is the correct place for the weakness to sit. No adversarial red-team exercise was conducted, and until one is, §6.5 is an argument from construction rather than a measured result.

6.6 Known limitations and dormant mechanics

Cross-night recurrence is implemented but never fires. `ConsequenceManager` scans each night's history for tickets closed with the symptom cleared but the root cause unrepaired, schedules a recurrence, and `RecurringProblemFactory` prefers due recurrences when generating tickets. Every wire is connected. It nonetheless never triggers, because the content generator emits `symptomCleared` and `rootCauseFixed` as *identical* condition sets, so no ticket can satisfy one without the other. The mechanic is a fully built road with no traffic. This is a content gap, not a code gap: authoring even a handful of faults whose two conditions genuinely differ — a printer whose queue is cleared without addressing why it filled, for instance — would activate the game's only cross-night consequence and, with it, the incident-versus-problem lesson of §4.7. It is the highest-value low-cost improvement available.

Scoring is coarse. The linear formula distinguishes only resolved from degraded, so a player who solves a fault in two precise actions scores identically to one who tries everything until something works. Since the design's thesis is that *reasoning* is the skill, the absence of feedback on reasoning *quality* is a substantive mismatch between the design's values and its measurements.

Voice was descoped and remains so. The dialogue layer is input-agnostic, so speech recognition and synthesis attach at the interface without redesign.

The persona trust trait exists but is unused. The ledger reserves a field intended to modulate customer tone across nights; nothing reads it.

Generated assets lag the generator (§6.3), a one-command fix left visible here for accuracy.

6.7 Threats to validity

Four threats bear on the conclusions of this chapter.

Construct validity. The artefact is evaluated against its own specification, and the specification was written by the same author. A mechanism can be present, correct and pedagogically useless.

Internal validity. Cascade verification asserts by human reading of forty-one log lines. A subtle regression in a reason string, or in a status on a module the reader is not focused on, could pass unnoticed.

External validity. The domain model is a stylised single-lane POS installation. Real installations have multiple lanes, vendor-specific quirks and networks that fail in ways no clean cascade describes. Whether diagnostic skill developed here transfers to real service-desk work is unmeasured and unmeasurable from the present evidence.

Absence of user data. Nobody outside the development context has played the artefact under observation. Playability, comprehensibility, difficulty pacing and the actual leakiness of the conversational agent under adversarial play are all unknown, and the report should be read as claiming none of them.

7. Discussion

7.1 The artefact against the troubleshooting-instruction literature

Jonassen and Hung (2006) specify three knowledge types a troubleshooting environment must integrate and three requirements it must satisfy. Assessing the artefact against them directly:

Requirement (Jonassen and Hung, 2006)	Realisation	Assessment
Domain/conceptual knowledge	Knowledge base of forty articles; POS domain model with a real transaction lifecycle	Present, though thinner than a dedicated course would provide
Device knowledge (runnable model of <i>this</i> system)	Dependency graph with Blocked/Error propagation, reason strings pointing upstream, two out-of-cascade failure domains	Strongest element. The player is compelled to build a runnable model because the reason strings only make sense within one
Experiential/strategic knowledge	Forty faults organised by discriminability; five-tier progression; red herrings; risky repairs	Present by design; unverified in players
Generate and test a hypothesis for every action	Diagnostic actions reveal clues; fix actions have preconditions; risky actions require confirmation	Partially met — the game does not <i>require</i> the hypothesis to be articulated
Relate every action to a conceptual model	Blocked statuses name their upstream cause	Met, and mechanically enforced
Query an experienced troubleshooter	Knowledge base articles teaching discriminations, auto-attached then faded	Met asynchronously; no live expert or hint system

The one clear partial failure is instructive. Jonassen and Hung require hypothesis generation *before* each action; the artefact rewards it but never demands it, so a player can brute-force by running every diagnostic action in every application and reading the results. Nothing in the current design prevents exhaustive search, and the coarse scoring of §6.6 means nothing penalises it either. The two weaknesses compound: a design whose thesis is inference contains no mechanism that distinguishes inference from enumeration. The fix follows directly from stating the problem — a lightweight commitment device, in which the player names a suspected module before running a diagnostic action, and scoring rewards early correct commitment. That is a genuine design gap identified by holding the artefact against the literature, and it is the most valuable single finding of this evaluation.

Where the artefact goes beyond its source literature is in *sequencing by discriminability*. Jonassen and Hung's architecture concerns the structure of a troubleshooting environment; it says little about how to order a corpus of faults. This project's rule — that each fault must be separable from a fault already met, with the confusable neighbour and the distinguishing evidence recorded explicitly — turns content authoring into curriculum design. It is a small, transferable idea: the unit of content in a diagnostic game is not the fault but the *discrimination between two faults*, and a fault that adds no new discrimination adds no learning regardless of how much authoring it consumed.

7.2 Architecture as pedagogy

The strongest claim this project can make is that in a diagnostic game, architectural decisions *are* pedagogical decisions, and that the coupling runs both ways.

The clearest case is the Blocked/Error distinction. Read as software, it is a status enumeration with a propagation rule — unremarkable. Read pedagogically, it is the difference between an environment that teaches inference and one that teaches guessing. Mark a stopped print spooler as blocking and its clues are hidden, the player hits a wall and learns nothing except that walls exist; mark it as a local error on the Printer and the player learns the transferable lesson that a symptom's layer is not its cause's layer. One enumeration value, and the design either teaches or fails to. The specification's remark that this is the rule most easily violated is telling: it is easy to violate precisely because, as code, both choices look equally reasonable.

The same coupling holds for invariant P6 (assets are static, runtime state is separate). As software, it prevents editor state corruption. Pedagogically, it is what makes every ticket a clean instance of the same fault, which is the precondition for deliberate practice (Ericsson, Krampe and Tesch-Römer, 1993) — without it, faults would accumulate residue across sessions and no fault could be practised twice under identical conditions.

And it holds for the string-keyed state trade-off of §5.2. Sacrificing compile-time safety bought the ability to author forty faults as data. Since the corpus size determines whether the discriminability curriculum can exist at all, a decision that looks purely technical was in fact the enabling condition for the curriculum. Nystrom's (2014) warning about disproportionate indirection is worth holding alongside this: the trade-off paid off here because content volume was the binding constraint, and it would not pay off in a project with six faults.

7.3 Policy-as-brain: a pattern with reach beyond games

The dialogue architecture — a rule-based policy that decides content, a language model that only rewords it, a boundary object that withholds sensitive information, and a filter as backstop — generalises past this artefact, and it is worth stating in general terms because the constraint it solves is common.

The problem it solves is *information asymmetry under conversational pressure*: a system must converse naturally about a subject while withholding specific facts, against a user with an incentive to extract them. Games are one instance. Others include tutoring systems that must not give away an answer, clinical simulations where a simulated patient must not name their own diagnosis, customer-service training where the simulated caller must not know the correct procedure, and assessment where a conversational agent must not reveal marking criteria.

The prevailing engineering answer is guardrails: filter the output, constrain the topics (Rebedea *et al.*, 2023). The pattern here inverts the emphasis. Filtering treats containment as an adversarial problem with no completeness guarantee, since it must anticipate every phrasing of a leak. Withholding treats it as an architectural problem with a structural guarantee: no phrasing can reveal what was never provided. The three requirements are (1) a boundary object constructed by a single factory that copies only permitted fields, (2) a decision layer above the model that holds all rules the model must not be able to override, and (3) an ordering in which the safe output is produced first and the model's contribution is an *optional improvement* rather than a dependency.

The third requirement is the subtlest and, in practice, the most valuable. Because the template line is displayed first and overwritten only if the model answers in time, model latency, model failure, and model absence are all indistinguishable from the player's perspective — the game simply keeps working. Systems that await a model response before rendering inherit its latency and its outages; systems that render first and upgrade opportunistically do not. That ordering costs nothing and converts a hard dependency into an enhancement.

An honest caveat: the pattern buys its guarantee by *reducing* what the model can do. This customer cannot answer an unanticipated question interestingly, cannot develop across a campaign, and cannot surprise a designer. Peng *et al.* (2024) examine exactly the emergence that this architecture forecloses. That is the correct trade for a diagnostic game, where a surprising customer is a broken puzzle; it would be the wrong trade for a narrative game where emergence is the product. The pattern is not superior in general — it is superior when leakage is fatal.

7.4 Reflection on the specification-first method

The methodological choice of §3.2 produced effects worth reporting, both positive and negative.

It worked because the invariants were declared as invariants. Seven numbered principles gave every subsequent decision a stable test. The judgement "should a stopped spooler block the chain?" is unanswerable on aesthetics but answerable against P4 and against the Blocked/Error rule that follows from it. Design documents ordinarily decay because they record decisions without recording reasons, so a later reader cannot tell which parts are load-bearing. Separating invariants from decisions, and recording rejected alternatives inline, is what kept these documents authoritative through a full port to a different language.

Documenting rejected alternatives repaid its cost repeatedly. The record of *why* guidance matching is keyed on issue identifier rather than category (array-order dependence would misassign articles) is the kind of reasoning that is obvious when written and invisible six weeks later, when the "simplification" to category matching looks like an improvement.

Its cost is rigidity, and this bit. The specification prescribed managers as `MonoBehaviour` components. Implementation showed plain classes to be strictly better — explicit dependencies, no inspector misconfiguration, testable without a scene — so the specification was overridden and the deviation documented. That is the correct outcome, but it required a conscious decision to depart from a document treated as authoritative, and a less confident developer might have implemented the worse design out of deference. A specification is a tool, and its authority must be conditional on continuing to be right.

Prototype-then-port was the right sequence. The Blocked/Error distinction, the terminal-identity split between P6 and P7, and the four-stage staff login were all discovered in JavaScript at a fraction of what the same discoveries would have cost in a half-built Unity scene. The duplicated effort bought a port that was translation rather than exploration, and an oracle against which disagreements could be diagnosed as porting errors rather than open design questions.

7.5 Practical and ethical considerations

Three points deserve brief treatment.

Local inference as a privacy position. Running the language model locally through Ollama means no player utterance leaves the machine and no API cost accrues. Given that the game invites free-text typing, cloud inference would place arbitrary player text on a third-party service, and the local choice avoids that entirely rather than mitigating it with policy.

Model choice and volatility. The specification recommends a small English-first instruct model in the ~ 1.5 –3B class and explicitly warns that names and versions change quickly, advising verification of the current best option and its commercial licence before shipping. The architecture backs this up: the model is one interface implementation, configured by string, and swapping it requires no code change.

Representation of the customer. The design's premise is that the customer is non-technical, misdescribes, and sometimes lies. There is a real risk of this becoming contempt — training an agent to view users as obstacles. Two design decisions push against it. The misnaming mechanic is framed as a *vocabulary* difference rather than a *competence* deficit: calling a terminal "the card machine" is a reasonable thing for a shopkeeper to do. And the "ask the customer to reseal the cable" family of repairs positions the customer as a collaborator with hands in the room the agent cannot reach — not everything on a counter is reachable over remote access, and pretending otherwise would teach a worse reflex than any misnaming. Whether the tone succeeds is a question for playtesting, and it is not answered here.

8. Conclusion and Future Work

8.1 Conclusion

This project set out to build a simulation game that develops diagnostic reasoning for POS technical support, with faults represented so that diagnosis requires inference, and with a language-model customer that is convincingly unhelpful without leaking the solution. Against the six objectives of §1.3:

O1 (domain model) — achieved. Six coupled modules propagate faults through an explicit dependency graph, with the Blocked/Error distinction correctly separating upstream-caused failures from local ones, and with two failure domains — per-staff login and database connectivity — deliberately placed outside the cascade. The distinction is verified live in the implementation for the cases most likely to violate it (§6.2).

O2 (fault taxonomy) — achieved. Forty faults are authored under an explicit discriminability rule, each recording the fault it is confusable with and the evidence that separates them. The audit of §6.4 confirms every fault has an observable discriminator, including three pairs that produce identical error text from different causes. The rule was subsequently extended from faults to identity: the CRM directory is generated in families of mistakeable shop names rather than authored one record at a time, and each site's network identity is derived from its own store name, which is what makes a single authored fault a correct, fixable fault for every shop in the directory (§4.8, §5.3).

O3 (architecture) — achieved. A four-layer architecture separates authored assets from runtime state; verdicts are pure functions of current state; Factory Method separates fault-selection sources from ticket assembly, demonstrated when recurrence was added as a decorator without modifying either existing factory.

O4 (contained agent) — achieved, with unmeasured residual risk. The dialogue pipeline withholds the root cause structurally rather than filtering for it, and two of the five containment layers are type-enforced rather than behavioural. Because the model can only reword a decided line, and because the safe line is rendered first, enabling the model can improve phrasing but can neither stall a call nor leak an answer. No adversarial evaluation was conducted.

O5 (verification) — partially achieved. Cascade verification over all forty faults passes and confirms the design's least intuitive rules. No automated test suite exists, assertion is by human reading, and multi-fault masking is exercised only in play. This is the artefact's clearest engineering deficiency.

O6 (critical evaluation) — achieved. Held against Jonassen and Hung (2006), the artefact satisfies the device-knowledge and conceptual-model requirements strongly, and fails one requirement in a way worth having found: it rewards hypothesis-before-action but never requires it, and its coarse scoring cannot distinguish inference from exhaustive search.

The broader contribution is a pair of transferable ideas rather than the artefact itself. The first is that in a diagnostic game the unit of content is the *discrimination between confusable faults*, not the fault, which makes content authoring a form of curriculum design. The second is *containment by construction*: when a conversational agent must not reveal a specific fact, withholding the fact from its context is a structurally stronger guarantee than filtering its output, and rendering a safe response first with the model as an optional improvement converts the model from a dependency into an enhancement.

The honest summary is that the artefact does what its specification says, that its architecture is sound and its content substantial, and that whether it *teaches* is unknown. Nothing in this report establishes a learning effect, and the design should be read as a well-argued hypothesis awaiting the test set out below.

8.2 Future work

Immediate, low cost, high value

1. **Automated test suite.** Assert the forty-one cascade readings against expected values rather than printing them; add unit tests for `ResolutionChecker`, `DependencyGraph`, `StaffLoginStatus`, `DbConnected`, and a statistical test that the refund/void case rate is approximately forty per cent. The layers are already plain C# and instantiable — the work is writing tests, not enabling them.
2. **Activate recurrence.** Author faults whose `symptomCleared` and `rootCauseFixed` conditions genuinely differ, so that the game's only cross-night consequence — and the incident-versus-problem lesson it carries — actually fires (§6.6).
3. **Regenerate content assets** so the committed corpus matches the generator — required, not cosmetic, since the store-named values in three assets predate tokenisation (§6.3). Regeneration reuses each existing asset file rather than recreating it, so scene references survive.
4. **Assert the store-agnostic corpus.** A generator-side check that no authored fault value or resolution condition contains a literal SSID or record host would make the class of bug in §6.3 impossible to reintroduce: the rule is that store-named content must be a token.
5. **Multi-fault harness coverage.** Extend the smoke test to blocker-over-fault combinations and assert the `Latent` → `Active` promotion, the one mechanic whose correctness currently rests on manual play.

Design work

6. **Hypothesis commitment.** Require the player to name a suspected module before running a diagnostic action, and reward early correct commitment. This closes the one requirement of Jonassen and Hung's (2006) architecture the artefact fails (§7.1) and simultaneously gives scoring something meaningful to measure.
7. **Reasoning-quality scoring.** Extend the score breakdown to root-cause correctness, redundant actions, time taken, and temporary-versus-permanent repair, so that feedback reflects the skill the design claims to teach.
8. **Debrief screen.** Garris, Ahlers and Driskell (2002) hold that game-based learning requires debriefing to convert experience into learning. An end-of-night review showing, per ticket, the actual fault, the actual dependency chain, and the shortest diagnostic path would add the reflective observation stage of Kolb's (1984) cycle, which the artefact currently omits.

Empirical evaluation

9. **Adversarial containment study.** Recruit participants, including technically sophisticated ones, and instruct them explicitly to extract the root cause from the customer through any means, with the language model enabled. Record every leak. This converts §6.5 from an argument from construction into a measured result.
10. **Learning-effect study.** A between-subjects design with a diagnostic pre-test, a fixed play period, an immediate post-test and a delayed retention test — measuring, in particular, transfer to fault *pairs* the par-

ticipant did not encounter, since discriminability is the design's central claim. Wouters *et al.* (2013) supply comparable effect sizes for calibration and Sweetser and Wyeth's (2005) GameFlow model supplies an experience instrument.

11. **Playability and pacing.** Structured observation with think-aloud protocol to test whether the interface is comprehensible without instruction and whether the five-tier progression matches actual competence growth.

Extensions

12. **Voice (M7).** Attach speech recognition and synthesis to the already input-agnostic dialogue layer, which would substantially increase the fidelity of the "noisy human sensor" premise.
13. **On-device intent classification.** Replace the keyword classifier with a small Sentis model to close the residual gap in §6.5, where an unanticipated phrasing of a technical question falls through to a confused reply.
14. **Multi-lane and multi-store installations,** raising the ceiling on domain fidelity noted in §6.7.

9. References

- Abt, C.C. (1970) *Serious Games*. New York: Viking Press.
- Anderson, J.R. (1982) 'Acquisition of cognitive skill', *Psychological Review*, 89(4), pp. 369–406.
- AXELOS (2019) *ITIL Foundation: ITIL 4 Edition*. London: TSO.
- Brooke, J. (1996) 'SUS: a "quick and dirty" usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189–194.
- Chi, M.T.H., Feltovich, P.J. and Glaser, R. (1981) 'Categorization and representation of physics problems by experts and novices', *Cognitive Science*, 5(2), pp. 121–152.
- Clark, D.B., Tanner-Smith, E.E. and Killingsworth, S.S. (2016) 'Digital games, design, and learning: a systematic review and meta-analysis', *Review of Educational Research*, 86(1), pp. 79–122.
- Csikszentmihalyi, M. (1990) *Flow: The Psychology of Optimal Experience*. New York: Harper & Row.
- de Kleer, J. and Williams, B.C. (1987) 'Diagnosing multiple faults', *Artificial Intelligence*, 32(1), pp. 97–130.
- Endsley, M.R. (1995) 'Toward a theory of situation awareness in dynamic systems', *Human Factors*, 37(1), pp. 32–64.
- Ericsson, K.A., Krampe, R.T. and Tesch-Römer, C. (1993) 'The role of deliberate practice in the acquisition of expert performance', *Psychological Review*, 100(3), pp. 363–406.
- Evans, E. (2003) *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley.
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Fullerton, T. (2014) *Game Design Workshop: A Playcentric Approach to Creating Innovative Games*. 3rd edn. Boca Raton: CRC Press.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Garris, R., Ahlers, R. and Driskell, J.E. (2002) 'Games, motivation, and learning: a research and practice model', *Simulation & Gaming*, 33(4), pp. 441–467.
- Gee, J.P. (2003) *What Video Games Have to Teach Us About Learning and Literacy*. New York: Palgrave Macmillan.
- Gentner, D. and Stevens, A.L. (eds.) (1983) *Mental Models*. Hillsdale, NJ: Lawrence Erlbaum Associates.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023) 'Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection', *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. New York: ACM, pp. 79–90.
- Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

- Hipple, R. (2017) *Game architecture with Scriptable Objects*. Unite Austin 2017. Available at: <https://github.com/roboryantron/Unite2017> (Accessed: 4 August 2026).
- Hunicke, R., LeBlanc, M. and Zubek, R. (2004) 'MDA: a formal approach to game design and game research', *Proceedings of the AAAI Workshop on Challenges in Game AI*. San Jose: AAAI Press, pp. 1–5.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), Article 248.
- Jonassen, D.H. and Hung, W. (2006) 'Learning to troubleshoot: a new theory-based design architecture', *Educational Psychology Review*, 18(1), pp. 77–114.
- Klein, G. (1998) *Sources of Power: How People Make Decisions*. Cambridge, MA: MIT Press.
- Kolb, D.A. (1984) *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice Hall.
- Koster, R. (2013) *A Theory of Fun for Game Design*. 2nd edn. Sebastopol, CA: O'Reilly Media.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474.
- Malone, T.W. (1981) 'Toward a theory of intrinsically motivating instruction', *Cognitive Science*, 5(4), pp. 333–369.
- Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.
- Michael, D. and Chen, S. (2006) *Serious Games: Games That Educate, Train, and Inform*. Boston: Thomson Course Technology.
- Night Signal Entertainment (2024) *Home Safety Hotline* [Video game]. Night Signal Entertainment.
- Norman, D.A. (1983) 'Some observations on mental models', in Gentner, D. and Stevens, A.L. (eds.) *Mental Models*. Hillsdale, NJ: Lawrence Erlbaum Associates, pp. 7–14.
- Nystrom, R. (2014) *Game Programming Patterns*. Genever Benning.
- Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P. and Bernstein, M.S. (2023) 'Generative agents: interactive simulacra of human behavior', *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. New York: ACM, Article 2.
- Peppers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2007) 'A design science research methodology for information systems research', *Journal of Management Information Systems*, 24(3), pp. 45–77.
- Peng, X., Quaye, J., Rao, S., Xu, W., Botchway, P., Brockett, C., Jojic, N., DesGarennes, G., Lobb, K., Xu, M., Leandro, J., Jin, C. and Dolan, B. (2024) 'Player-driven emergence in LLM-driven game narrative', *2024 IEEE Conference on Games (CoG)*. Milan: IEEE.
- Pope, L. (2013) *Papers, Please* [Video game]. 3909 LLC.
- Prensky, M. (2001) *Digital Game-Based Learning*. New York: McGraw-Hill.

- Rasmussen, J. (1983) 'Skills, rules, and knowledge; signals, signs, and symbols, and other distinctions in human performance models', *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(3), pp. 257–266.
- Reason, J. (1990) *Human Error*. Cambridge: Cambridge University Press.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C. and Cohen, J. (2023) 'NeMo Guardrails: a toolkit for controllable and safe LLM applications with programmable rails', *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Singapore: Association for Computational Linguistics, pp. 431–445.
- Reiter, R. (1987) 'A theory of diagnosis from first principles', *Artificial Intelligence*, 32(1), pp. 57–95.
- Ryan, R.M., Rigby, C.S. and Przybylski, A. (2006) 'The motivational pull of video games: a self-determination theory approach', *Motivation and Emotion*, 30(4), pp. 344–360.
- Salen, K. and Zimmerman, E. (2004) *Rules of Play: Game Design Fundamentals*. Cambridge, MA: MIT Press.
- Schell, J. (2019) *The Art of Game Design: A Book of Lenses*. 3rd edn. Boca Raton: CRC Press.
- Schön, D.A. (1983) *The Reflective Practitioner: How Professionals Think in Action*. New York: Basic Books.
- Sitzmann, T. (2011) 'A meta-analytic examination of the instructional effectiveness of computer-based simulation games', *Personnel Psychology*, 64(2), pp. 489–528.
- Sweetser, P. and Wyeth, P. (2005) 'GameFlow: a model for evaluating player enjoyment in games', *Computers in Entertainment*, 3(3), pp. 1–24.
- Sweller, J. (1988) 'Cognitive load during problem solving: effects on learning', *Cognitive Science*, 12(2), pp. 257–285.
- Tendershoot (2019) *Hypnospace Outlaw* [Video game]. No More Robots.
- Unity Technologies (2025) *Unity Manual: ScriptableObject*. Available at: <https://docs.unity3d.com/Manual/class-ScriptableObject.html> (Accessed: 4 August 2026).
- Weidinger, L., Mellor, J., Rauh, M., Griffin, C., Uesato, J., Huang, P-S., Cheng, M., Glaese, M., Balle, B., Kasirzadeh, A., Kenton, Z., Brown, S., Hawkins, W., Stepleton, T., Biles, C., Birhane, A., Haas, J., Rimell, L., Hendricks, L.A., Isaac, W., Legassick, S., Irving, G. and Gabriel, I. (2021) 'Ethical and social risks of harm from language models', *arXiv preprint arXiv:2112.04359*.
- Wouters, P., van Nimwegen, C., van Oostendorp, H. and van der Spek, E.D. (2013) 'A meta-analysis of the cognitive and motivational effects of serious games', *Journal of Educational Psychology*, 105(2), pp. 249–265.

10. Appendices

Appendix A — The forty-fault corpus

#	Faulty state	Category	Confusable with	Discriminating evidence
P1	<code>Printer.paperLevel = Empty</code>	Printer	—	Queue reports "out of paper"
P2	<code>Printer.driverState = Corrupted</code>	Printer	P3	Device Manager error 39; queue stuck
P3	<code>CashDrawer.port = COM3</code>	CashDrawer	P2	Driver healthy; port collision
P4	<code>Network.isOnline = false</code>	Network	—	Blocker: whole chain unusable
P5	<code>POSSoftware.receiptTemplate = Broken</code>	POS	Printer faults	Test page fine; customer copy missing fields
P6	<code>Terminal.wifiNetwork ≠ store SSID</code>	Terminal	P7	Terminal on wrong SSID; IP range differs
P7	<code>POSSoftware.registeredTerminalIp stale</code>	POS	P6	Terminal on correct SSID; POS holds an old IP
P8	<code>POSSoftware.staffRole = None</code>	Business	Hardware faults	One user only; cards still charge
P9	<code>POSSoftware.staffTerminal = ""</code>	Business	P8	Role present, no assignment at all
P10	<code>staffTerminal = REG-4 ≠ machineId</code>	Business	P9	Assigned, but to another register
P11	<code>terminalSynced = false</code>	Business	P8–P10	Role and assignment correct; not yet synced
P12	<code>POSSoftware.dbHost mistyped</code>	POS	P36	Lookup fails; printing fine
P13	<code>OS.spoolerService = Stopped</code>	Printer*	P1, P2	Paper present, driver fine, jobs "Spooling"
P14	<code>OS.diskSpace = Full</code>	OS	—	Blocker: nothing runs
P15	<code>OS.pendingReboot = true</code>	OS	—	Blocker; risky fix drops the session
P16	<code>OS.systemTime = Skewed</code>	Terminal*	P6, P7, P37	Network fine, cash fine, all cards declined
P17	<code>Printer.paperJam = Jammed</code>	Printer	P1	Paper present plus mechanical noise
P18	<code>Printer.cableConnected = false</code>	Printer	P2	Device not listed at all
P19	<code>Printer.queuePaused = true</code>	Printer	P13	Paused ≠ no service running
P20	<code>Printer.defaultPrinter = OfficeInkjet</code>	Printer	P13, P19	Output appears on another printer
P21	<code>POSSoftware.printerVisible = false</code>	POS	P20	Windows sees it, POS does not
P22	<code>Printer.connection = Offline</code>	Printer	P2, P18	"Use printer offline" was ticked

#	Faulty state	Category	Confusable with	Discriminating evidence
P23	<code>Printer.paperWidth = 58mm</code>	Printer	P5	Content complete but truncated
P24	<code>CashDrawer.lockState = Locked</code>	CashDrawer	P3	Audible click: power reached it
P25	<code>CashDrawer.triggerMode = Manual</code>	CashDrawer	P3, P24	"Must press by hand" ≠ "won't open"
P26	<code>Terminal.pairingState = Unpaired</code>	Terminal	P6, P7	Network and IP correct; pairing token lost
P27	<code>Terminal.firmwareVersion = 3.1</code>	Terminal	P26	Broke immediately after an update
P28	<code>Terminal.emvConfig = Corrupt</code>	Terminal	P6, P37	Chip fails, swipe works
P29	<code>Terminal.mode = Training</code>	Terminal	P32	Everything approves; no money arrives
P30	<code>POSSoftware.licenseState = Expired</code>	POS	P38	Application refuses to start
P31	<code>POSSoftware.offlineMode = true</code>	POS	P4	Network healthy; POS chose offline
P32	<code>POSSoftware.batchState = SettleFailed</code>	POS	P29	Transactions exist but were never sent
P33	<code>POSSoftware.taxRate = 0</code>	POS	P5	Field present, value wrong
P34	<code>POSSoftware.priceSync = Stale</code>	POS	P33	Connection healthy, price list old
P35	<code>Network.signalStrength = Weak</code>	Network	P4	Fails only under load
P36	<code>Network.dnsServer = 8.8.8.8</code>	Network	P12	Correct host name, nothing resolves it
P37	<code>Network.firewallBlocking = true</code>	Network	P16	Internet fine, cards dead — check clock first
P38	<code>OS.antivirusQuarantine = true</code>	OS	P30	Began after a security alert
P39	<code>OS.userAccount = Standard</code>	OS	P8	Windows permissions, not POS role
P40	<code>OS.powerPlan = Sleep</code>	OS	P26, P35	Only when idle; awake once connected

* Fault lives in the OS module but is filed under the category where the player will look for it.

Appendix B — Modules and application mapping

Module	Dependency	Representative state fields	Applications exposing it
OS	root	<code>diskSpace</code> , <code>pendingReboot</code> , <code>spoolerService</code> , <code>systemTime</code> , <code>antivirusQuarantine</code> , <code>userAccount</code> , <code>powerPlan</code>	System (health, services)
Network	OS (machine-wide only)	<code>isOnline</code> , <code>ssid</code> , <code>signalStrength</code> , <code>dnsServer</code> , <code>firewallBlocking</code>	Network Settings

Module	Dependency	Representative state fields	Applications exposing it
POSSoftware	Network (down only)	receiptTemplate, registeredTerminalIp, dbHost, staffRole, staffTerminal, terminalSynced, printerVisible, licenseState, offlineMode, batchState, taxRate, priceSync	POS Manager (receipt, connections, staff, database)
Terminal	POSSoftware	wifiNetwork, machineId, pairingState, firmwareVersion, emvConfig, mode	POS Terminal (status, batch)
Printer	POSSoftware	paperLevel, driverState, connection, paperJam, cableConnected, queuePaused, defaultPrinter, paperWidth, port	Printer & Print Queue; Device Manager
CashDrawer	Printer	port, lockState, triggerMode	Cash Drawer Config

Appendix C — Services and single responsibilities

Service	Exclusive privilege
CampaignManager	Owns campaign-scoped state; sole caller of persistence
ShiftManager	Owns the night clock and spawn tempo
TicketManager	Sole authority on call lifecycle status
ProblemGenerator	Only place a problem instance is created
VerificationManager	CRM lookup, click-to-compare, remote connection
StoreDirectoryFactory	Only source of CRM accounts; only place a site identity is derived
DesktopManager	Builds desktops; applies state changes; promotes latent faults
ActionManager	All player actions on the simulated desktop
ResolutionChecker	Sole authority on health verdicts (static, pure)
TransactionManager	Batch and history lifecycle; authorisation gate
DialogueManager	The only producer of customer speech
CommunicationManager	Channel routing; SMS receipt mechanic
MailboxManager	Only creator of complaints and night-failure state
ConsequenceManager	Only cross-night consequence mechanism
ScoreManager	Only computer of currency
KnowledgeBaseManager	Article lookup and guidance matching
SaveManager	Only code that touches storage

Appendix D — Artefact metrics

Metric	Value
C# source	7,619 lines across 50 files
Design specification	1,465 lines across 4 documents

Metric	Value
Web prototype (reference)	≈ 2,700 lines (1,969 JavaScript)
Authored faults	40 (P1–P40)
Knowledge-base articles	40 (KB-001–KB-040)
Desktop actions	50
Simulated modules	6 in cascade
Simulated applications	7
Confusable name families authored	16 (6 rolled per campaign)
CRM accounts per campaign	≈ 20, in clusters of 3–4
Diagnostic action types	17
Player intents	14
Dialogue act kinds	13
Banned technical terms	≈ 50
Enumerations	20
Services	15 + 1 static
Engine	Unity 6000.5.4f1, URP 17.5.0, uGUI 2.5.0, Input System 1.19.0, Sentis 2.6.1
Default language model	llama3.2:3b via local Ollama (opt-in)
Shift length	480 s real time = 20:00–04:00 in-game (8 in-game hours)
Call volume	Configured rate: 0.25 calls/in-game hour +0.00625 per day, clamped 1–12 per night
Campaign	60 nights; ≥ 150 tickets to win; 3 complaints fail a night; 3 warnings end the campaign

Appendix E — Glossary

Term	Meaning in this project
Blocked	A module is healthy but cannot reach an upstream dependency; its clues are hidden
Error	A module is itself faulty; its clues must remain readable
Latent fault	A fault injected behind a blocker: unobservable, unrepairable, ungraded
MadeWorse	A player action introduced a fault that was not originally present
Degraded	Ticket verdict when harm occurred, technical or business
GroundTruth	The reduced view of a ticket handed to the AI layer; the containment boundary
DialoguePolicy	The rule layer deciding what the customer may say ("the brain")
GroundingGuard	Output filter checking jargon and leaked state field names ("the backstop")
Discriminability rule	Every authored fault must be separable from a fault the player already knows
Misnaming	Persona-driven substitution of correct device names with lay terms

Term	Meaning in this project
Recurrence	A symptom-cleared but root-cause-unrepaired fault returning on a later night
Name cluster	A family of mistakeable first words ({Sunrise, Sunset, Sunnyside}) crossed with trades to build confusable CRM accounts
StoreIdentity	A shop's derived site identity: Wi-Fi SSIDs and record-store host, computed from its name
Token	A placeholder in authored content ({SSID} , {DB_HOST}) resolved against the calling shop's identity

End of report.